

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Алтайский государственный технический университет им. И.И. Ползунова»

Факультет (институт) ФИТ

Кафедра прикладной математики

Отчет защищен с оценкой _____
Зав. Кафедрой Боровцов Е.Г.
(подпись) (Фамилия И.О.)

Отчёт по практике

Вид	Учебная практика
-----	------------------

Код и наименование направления подготовки (специальности): _____
09.03.04 Программная инженерия

Направленность (профиль, специализация): _____
Разработка программно-информационных систем

Форма обучения: очная

Студента Панкрашина Ярослава Евгеньевича
(Фамилия Имя Отчество)

Группа ПИ-43

г. Барнаул

ФГБОУ ВО «Алтайский государственный технический университет
им. И. И. Ползунова»
Кафедра прикладной математики

Индивидуальное задание

на учебную практику

(вид и тип практики по УП)

студенту Панкрашину Ярославу Евгеньевичу
(Ф.И.О.)

группы ПИ-43

График проведения практики:

№п/п	Содержание работ, выполняемых на практике	Сроки выполнения
1	Получение индивидуального задания на практику. Прохождение инструктажа по охране труда и ТБ. Изучение предметной области и формирование технического задания.	1 неделя
2	Разработка и документирование архитектуры программного продукта	2 неделя
3	Конструирование, отладка и тестирование программного продукта.	2-4 неделя
4	Оформление и защита отчёта о практике.	4 неделя

Руководитель практики от университета _____
(подпись)

Андреева А.Ю., доцент
(Ф.И.О., должность)

Задание принял к исполнению _____
(подпись)

Панкрашин Я.Е.
(Ф.И.О.)

Инструктаж по ОТ, ТБ, ПБ, ПВТР

Инструктаж обучающегося по ознакомлению с требованиями охраны труда, техники безопасности, пожарной безопасности, а также правилами внутреннего трудового распорядка проведён «___» _____ 20___ г.

Руководитель практики от
Университета

(подпись)

Андреева А.Ю., доцент
(Ф.И.О., должность)

Аннотация

В данной работе представлена имитационная модель системы "Хищник-жертва", реализованная в виде веб-приложения на платформе React с использованием TypeScript. Моделируется взаимодействие двух популяций: травоядных (Prey) и хищников (Predator) в замкнутой среде с ограниченными ресурсами. Животные имеют стадии развития (молодой/взрослый/старый), параметры энергии и возраста, а также способность к размножению и смерти от голода или старости. В модель добавлен фактор окружающей среды - смена сезонов года, влияющая на скорость восстановления растительности. Приложение обеспечивает визуализацию процесса, отображение статистики, лог событий и интерактивное получение информации о каждом объекте. Реализована с использованием объектно-ориентированного подхода с наследованием и полиморфизмом

Содержание

Введение.....	7
1. Техническое задание.....	8
1.1 Цель работы.....	8
1.2 Требования к функциональности	8
1.3 Требования к интерфейсу.....	9
1.4 Требования к программной реализации.....	10
2. Проект программного продукта.....	11
2.1 Архитектура приложения.....	11
2.2 Диаграмма классов.....	12
2.3 Структура данных.....	13
2.4 Алгоритмы взаимодействия объектов.....	13
3. Описание программного продукта.....	16
3.1 Начало работы.....	16
3.2 Панель управления.....	16
3.3 Панель настроек.....	16
3.4 Панель статистики.....	16
3.5 Лог событий.....	17
3.6 Визуализация симуляции.....	17
3.7 Интерактивное взаимодействие.....	17
Заключение	19
Список используемых источников.....	20
Приложение А. Снимки экранных форм пользовательского интерфейса.....	21
Приложение Б. Тестирование.....	23
Приложение В. Исходный код программы.....	24

Введение

Имитационное моделирование является одним из наиболее эффективных методов исследования сложных динамических систем, когда прямое экспериментирование с реальным объектом затруднено или невозможно. Особенно актуально применение имитационного моделирования в биологии и экологии для изучения популяционной динамики, где временные масштабы процессов могут составлять годы и десятилетия.

Классическая модель "хищник-жертва" (модель Лотки-Вольтерры) описывает циклические колебания численности двух взаимодействующих популяций. Однако аналитическая модель не учитывает множество факторов реального мира: индивидуальные характеристики особей (возраст, энергию, стадию развития), пространственное распределение, влияние окружающей среды.

Целью данной работы является разработка имитационной модели системы "хищник-жертва" с дискретными агентами (agent-based modeling), где каждое животное представлено отдельным объектом со своими параметрами и поведением. Такой подход позволяет:

- наблюдать за судьбой каждого отдельного организма;
- анализировать влияние индивидуальных характеристик на популяционную динамику;
- изучать эффекты от изменения параметров среды;
- получать наглядную визуализацию процессов.

В работе реализовано веб-приложение на современном стеке технологий (React + TypeScript), обеспечивающее:

- визуализацию перемещения и взаимодействия объектов;
- управление параметрами симуляции в реальном времени;
- ведение лога событий для последующего анализа;
- интерактивное получение информации о каждом объекте.

Данная работа может служить как учебным пособием по имитационному моделированию, так и основой для дальнейшего развития модели с добавлением новых факторов и видов.

1. Техническое задание

1.1 Цель работы

Разработать имитационную модель взаимодействия двух популяций (хищников и травоядных) в замкнутой среде с визуализацией процесса, возможностью управления параметрами симуляции и интерактивным получением информации об объектах модели.

1.2 Требования к функциональности

Программа должна обеспечивать выполнение следующих функций:

Моделирование популяции травоядных (Prey):

- Каждый объект имеет координаты (x, y), уровень энергии, возраст, статус (жив/мертв);
- Травоядные перемещаются к ближайшей доступной траве;
- При контакте с травой поедают её, восполняя энергию;
- Травоядные теряют энергию с течением времени;
- При достижении порога энергии и определенного возраста могут размножаться;
- Смерть наступает при истощении энергии или достижении максимального возраста.

Моделирование популяции хищников (Predator):

- Каждый объект имеет координаты (x, y), уровень энергии, возраст, статус;
- Хищники перемещаются к ближайшему травоядному;
- При контакте с травоядным поедают его, восполняя энергию;
- Хищники теряют энергию с течением времени;
- При достижении порога энергии и определенного возраста могут размножаться;
- Смерть наступает при истощении энергии или достижении максимального возраста.

Моделирование окружающей среды:

- На поле случайным образом распределены точки травы (150 штук);
- Съеденная трава восстанавливается через определенный интервал времени;
- Скорость восстановления травы зависит от текущего сезона года.

Стадии жизни:

- У каждого животного выделены 3 стадии: молодой (Young), взрослый (Adult), старый (Old);

- Стадия определяется возрастом объекта;
- От стадии зависят: скорость передвижения, размер отображения, цвет, способность к размножению.

Факторы среды:

- Реализована смена 4 сезонов года (весна, лето, осень, зима);
- Каждый сезон влияет на скорость восстановления травы;
- Визуальное отображение текущего сезона на поле симуляции.

Управление симуляцией:

- Кнопка "Start" - запуск симуляции;
- Кнопка "Pause" - приостановка симуляции;
- Кнопка "Reset" - сброс симуляции с текущими настройками.

Настройки симуляции:

- Регулировка количества травоядных (10-200);
- Регулировка количества хищников (1-50);
- Регулировка скорости травоядных (0.5-5);
- Регулировка скорости хищников (0.5-6).

Статистика и логирование:

- Отображение текущего сезона;
- Отображение времени симуляции;
- Отображение текущей численности популяций;
- Лог событий (рождения, смерти, поедания, смены сезонов);
- Возможность очистки лога.

Интерактивное взаимодействие:

- Клик по любому животному на поле отображает информацию о нем (ID, тип, энергия, возраст, стадия, скорость, размер, статус).

1.3 Требования к интерфейсу

- Интерфейс должен быть интуитивно понятным;

- Визуализация должна наглядно отображать процесс симуляции;
- Различные типы объектов должны иметь различные цвета и размеры;
- Цвет объектов должен меняться в зависимости от стадии жизни и уровня энергии;
- Должна отображаться информация о текущем сезоне;
- Лог событий должен иметь цветовую кодировку типов событий;
- Интерфейс не должен иметь скроллов, все элементы располагаются на одном экране.

1.4 Требования к программной реализации

- Язык программирования: TypeScript;
- Фреймворк: React;
- Стилизация: inline styles / CSS;
- Архитектура: компонентный подход с разделением на модели, хуки и компоненты;
- Использование объектно-ориентированного подхода (классы, наследование, полиморфизм);
- Паттерн Singleton для системы логирования;
- Частота обновления симуляции: ~30 кадров в секунду;
- Размер поля симуляции: 800×600 пикселей.

2. Проект программного продукта

2.1 Архитектура приложения

Приложение построено на основе компонентной архитектуры React с четким разделением на уровень представления (UI) и уровень модели (бизнес-логики).

Уровень представления (Components) включает следующие компоненты:

- App.tsx - корневой компонент, организующий общую компоновку интерфейса и управляющий состоянием настроек (количество животных, скорость).
- SimulationCanvas.tsx - отвечает за отрисовку симуляции на canvas и обработку кликов мыши для получения информации об объектах.
- ControlPanel.tsx - предоставляет кнопки управления (Start, Pause, Reset).
- SettingsPanel.tsx - предоставляет ползунки для настройки параметров симуляции.
- StatisticsPanel.tsx - отображает текущую статистику (время, численность популяций, сезон).
- EventLogPanel.tsx - отображает лог событий в реальном времени.
- ObjectInfo.tsx - всплывающее окно с информацией о выбранном объекте.

Уровень хуков (Hooks) представлен кастомным хуком useSimulation.ts, который управляет жизненным циклом симуляции: запуск, пауза, сброс. Хук создает экземпляр класса World и запускает интервал обновления.

Уровень модели (Models) включает следующие классы:

- World.ts - главный класс, содержащий все массивы объектов (травы, травоядные, хищники) и управляющий их обновлением на каждом тике.
- Animal.ts - абстрактный базовый класс, содержащий общую логику для всех животных (движение, возраст, энергия, стадии жизни).
- Prey.ts и Predator.ts - классы-наследники Animal, реализующие специфическое поведение (поиск травы или поиск добычи).
- Grass.ts - класс, представляющий растение (траву) с логикой восстановления после поедания.
- SeasonManager.ts - класс, управляющий сменой сезонов и коэффициентами влияния на рост травы.

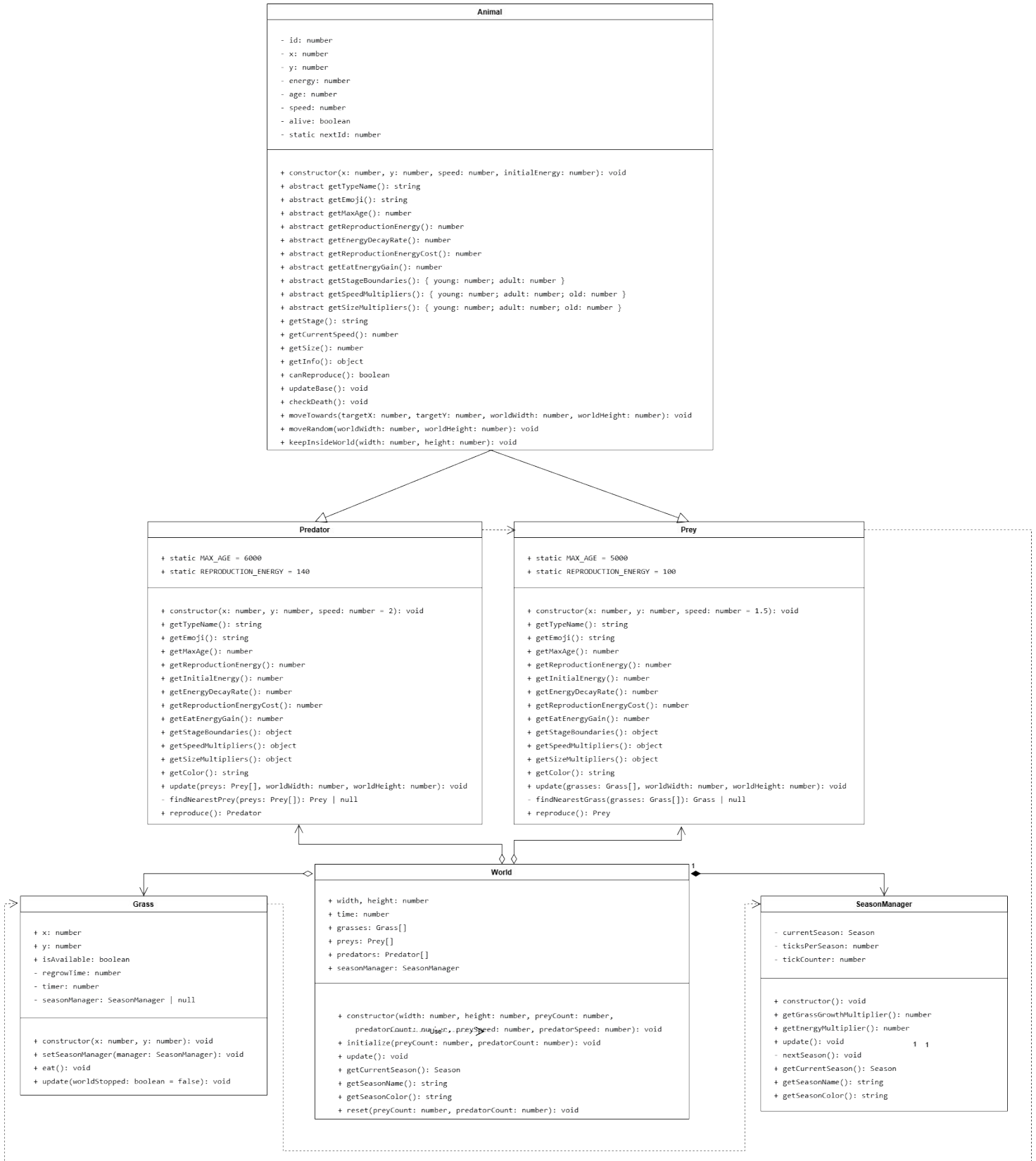
Уровень утилит (Utils) представлен классом EventLog.ts, реализованным в виде Singleton для централизованного сбора и хранения событий, происходящих в симуляции.

Взаимодействие между уровнями организовано следующим образом:

- Компоненты через пропсы получают данные из хука useSimulation.
- Хук useSimulation создает и хранит экземпляр World.
- World управляет массивами Prey, Predator, Grass и делегирует им обновление.
- Prey и Predator наследуют общую логику от Animal.

- При любых изменениях (рождение, смерть, поедание) объекты обращаются к EventLog для записи события.
- World также содержит экземпляр SeasonManager, который влияет на скорость восстановления травы.

2.2 Диаграмма классов



2.3 Структура данных

Формат LogEvent:

```
{  
    id: number;      // уникальный номер события  
    timestamp: number; // время события (Unix timestamp)  
    message: string;  // текст события  
    type: "birth" | "death" | "eat" | "season" | "info"  
}
```

Формат информации об объекте:

```
{  
    id: number;      // ID объекта  
    type: string;    // тип (с эмодзи)  
    energy: number;  // текущая энергия  
    age: number;     // возраст в тиках  
    ageYears: number; // возраст в условных годах  
    stage: string;   // стадия (Young/Adult/Old)  
    status: string;  // статус (alive/dead)  
    speed: string;   // текущая скорость  
    size: number;    // размер для отрисовки  
}
```

2.4 Алгоритмы взаимодействия объектов

Алгоритм движения травоядного (Prey):

1. Уменьшить энергию на скорость распада (0.05)
2. Увеличить возраст на 1
3. Найти ближайшую доступную траву:
 - Отфильтровать траву с `isAvailable = true`
 - Вычислить евклидово расстояние до каждой
 - Выбрать траву с минимальным расстоянием
4. Если трава найдена:

- Двигаться в сторону травы со скоростью (зависит от стадии)
- Если дистанция < 6 и трава доступна:
 - * Съесть траву (`isAvailable = false`)
 - * Увеличить энергию на 20

5. Если трава не найдена:

- Двигаться случайным образом

6. Проверить условия смерти:

- Если энергия ≤ 0 : смерть от голода
- Если возраст $> \text{MAX_AGE}$: смерть от старости

Алгоритм движения хищника (Predator):

1. Уменьшить энергию на скорость распада (0.08)

2. Увеличить возраст на 1

3. Найти ближайшего живого травоядного:

- Отфильтровать травоядных с `alive = true`
- Вычислить евклидово расстояние до каждого
- Выбрать травоядного с минимальным расстоянием

4. Если травоядный найден:

- Двигаться в сторону травоядного со скоростью (зависит от стадии)
- Если дистанция < 8 и травоядный жив:
 - * Пометить травоядного как мертвого
 - * Увеличить энергию на 40

5. Проверить условия смерти:

- Если энергия ≤ 0 : смерть от голода
- Если возраст $> \text{MAX_AGE}$: смерть от старости

Алгоритм обновления травы:

1. Если трава доступна (`isAvailable = true`) - ничего не делать

2. Если трава съедена (`isAvailable = false`):

- Уменьшить таймер восстановления на 1
- Если таймер ≤ 0 :
 - * Установить `isAvailable = true`

- Время восстановления зависит от текущего сезона:

* Весна: быстрее ($\div 1.5$)

* Лето: нормально ($\div 1.2$)

* Осень: медленнее ($\div 0.8$)

* Зима: очень медленно ($\div 0.3$)

Алгоритм смены сезонов:

1. При каждом обновлении мира увеличивать счетчик тиков

2. Если счетчик достиг `ticksPerSeason` (600 тиков $\approx$ 18 секунд):

- Сбросить счетчик

- Переключить сезон: весна $\rightarrow$ лето $\rightarrow$ осень $\rightarrow$ зима $\rightarrow$ весна

- Записать событие в лог

3. Все параметры (скорость роста травы) автоматически обновляются

3. Описание программного продукта

3.1 Начало работы

При запуске приложения пользователь видит главное окно симуляции, разделенное на три области:

- Левая панель - статистика (текущий сезон, время симуляции, численность популяций)
- Центральная область - поле симуляции с визуализацией и заголовок с названием
- Правая панель - настройки параметров и лог событий

Симуляция запускается в приостановленном состоянии. Для начала работы необходимо нажать кнопку "START".

3.2 Панель управления

Панель управления расположена под полем симуляции и содержит три кнопки:

- START - запускает симуляцию. Кнопка становится неактивной во время работы.
- PAUSE - приостанавливает симуляцию. Активна только во время работы.
- RESET - останавливает симуляцию и сбрасывает мир к начальному состоянию с текущими настройками.

3.3 Панель настроек

Панель настроек расположена в правой части экрана и позволяет изменять параметры симуляции в реальном времени:

- Preys (10-200) - ползунок для регулировки начального количества травоядных
- Predators (1-50) - ползунок для регулировки начального количества хищников
- Prey speed (0.5-5) - ползунок для регулировки скорости травоядных
- Predator speed (0.5-6) - ползунок для регулировки скорости хищников

Все изменения применяются после нажатия кнопки RESET.

3.4 Панель статистики

Панель статистики расположена в левой части экрана и отображает:

- Текущий сезон года (с цветовым и эмодзи-обозначением)
- Текущее время симуляции в условных единицах
- Текущее количество травоядных
- Текущее количество хищников

3.5 Лог событий

Лог событий находится в правой части экрана под панелью настроек. Он отображает все значимые события симуляции:

- Рождение травоядного или хищника (зеленый цвет)
- Смерть от голода или старости (красный цвет)
- Поедание травоядного хищником (оранжевый цвет)
- Смена сезона (синий цвет)

Каждое событие содержит временную метку. Лог хранит последние 100 событий. Кнопка CLEAR позволяет очистить лог.

3.6 Визуализация симуляции

На поле симуляции отображаются:

- Трава - зеленые точки диаметром 3 пикселя. Доступная трава видна, съеденная временно исчезает.
- Травоядные - круги синего цвета. Размер и оттенок зависят от стадии жизни и уровня энергии.
- Хищники - круги красного цвета. Размер и оттенок зависят от стадии жизни и уровня энергии.
- ID объектов - отображаются над каждым животным.
- Индикатор энергии - цветная полоска под каждым животным, показывающая текущий уровень энергии.
- Текущий сезон - отображается в левом верхнем углу поля с соответствующим цветом и декоративными эффектами (снежинки зимой, цветы весной, листья осенью).

Цветовая дифференциация стадий жизни:

- Молодые (Young) - светлые оттенки
- Взрослые (Adult) - насыщенные оттенки
- Старые (Old) - темные оттенки

3.7 Интерактивное взаимодействие

При клике мышью на любое животное в правом нижнем углу экрана появляется информационное окно со следующими данными:

- Тип и ID животного
- Уровень энергии (с цветовой индикацией: зеленый - высокий, оранжевый - средний, красный - низкий)
- Возраст в условных годах и в тиках

- Текущая стадия жизни
- Текущая скорость
- Размер
- Статус (жив/мертв)

Окно закрывается повторным кликом на поле или нажатием кнопки "✕".

Заключение

В ходе выполнения работы была разработана имитационная модель системы "Хищник-жертва" в виде веб-приложения на платформе React с использованием TypeScript.

В результате выполненной работы были решены следующие задачи:

1. Разработана объектно-ориентированная модель взаимодействия двух популяций: травоядных (Prey) и хищников (Predator). Каждый объект имеет индивидуальные параметры (координаты, энергию, возраст, скорость) и автономное поведение.
2. Реализованы стадии жизни объектов (Young, Adult, Old), влияющие на скорость передвижения, размер отображения и возможность размножения. Только взрослые особи могут воспроизводить потомство.
3. Добавлен фактор окружающей среды - смена четырех сезонов года, влияющая на скорость восстановления растительности. Сезоны визуально отображаются на поле симуляции.
4. Реализована система логирования событий (рождения, смерти, поедания, смены сезонов) с использованием паттерна Singleton.
5. Разработан интерактивный интерфейс, позволяющий управлять симуляцией (Start, Pause, Reset), изменять параметры (количество и скорость объектов), получать подробную информацию о каждом объекте по клику мыши, а также наблюдать за текущей статистикой в реальном времени.
6. Обеспечена наглядная визуализация процесса: различные цвета и размеры для разных типов объектов и стадий жизни, индикаторы энергии, отображение ID, сезонные эффекты.
7. Продемонстрировано понимание принципов объектно-ориентированного программирования: создан абстрактный базовый класс Animal с наследованием и полиморфизмом, использован паттерн Singleton для системы логирования, применены принципы агрегации и композиции.

Программа полностью соответствует заявленным требованиям: объекты можно просмотреть индивидуально, за каждым показателем можно уследить через лог событий и информационные окна, визуализация является наглядной и понятной.

Разработанное приложение может служить основой для дальнейшего развития модели: добавления новых видов животных, дополнительных факторов среды (температура, влажность), построения графиков динамики популяций, а также масштабирования времени симуляции.

Список использованных источников

1. React Documentation. Main Concepts. URL: <https://react.dev> (дата обращения: 02.06.2026)
2. TypeScript Documentation. Handbook. URL: <https://www.typescriptlang.org/docs/> (дата обращения: 02.06.2026)
3. Лотка А. Дж. Элементы математической биологии. - 1925.
4. Вольтерра В. Математическая теория борьбы за существование. - 1931.
5. Гамма Э., Хелм Р., Джонсон Р., Влиссидес Дж. Приемы объектно-ориентированного проектирования. Паттерны проектирования. - СПб.: Питер, 2020. - 544 с.
6. Эккель Б. Философия Java. - СПб.: Питер, 2019. - 1168 с.

Приложение А. Снимки экранных форм пользовательского интерфейса

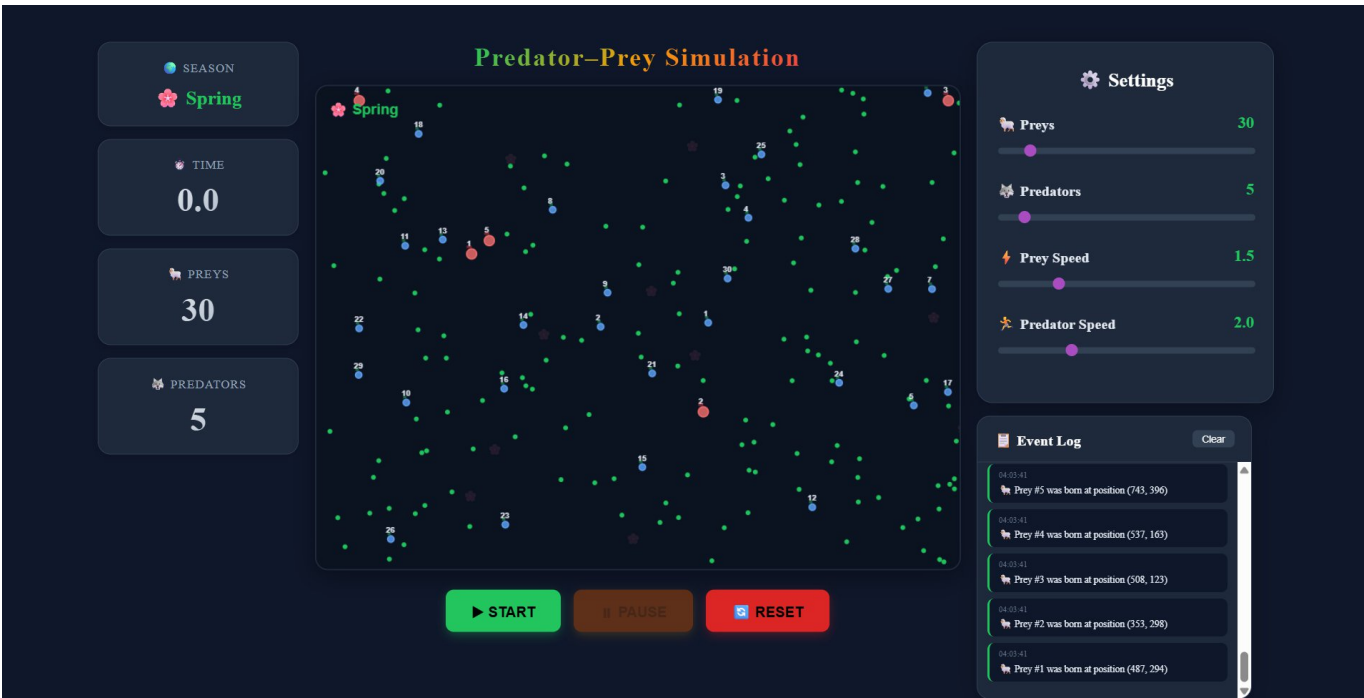


Рисунок 1. Главное окно приложения



Рисунок 2. Панель настроек



Рисунок 3. Лог событий



Рисунок 4. Информационное окно при клике на объект

Приложение Б. Тестирование

Таблица 1. План тестирования

№	Тестовый сценарий	Ожидаемый результат
1	Нажать START	Симуляция запускается, животные начинают движение
2	Нажать PAUSE	Симуляция приостанавливается
3	Нажать RESET	Симуляция сбрасывается, численность возвращается к настройкам
4	Изменить количество Prey на 100, нажать RESET	На поле появляется 100 травоядных
5	Изменить скорость Predator на 4, нажать RESET	Хищники двигаются быстрее
6	Кликнуть на любое животное	Появляется окно с информацией
7	Дождаться смены сезона	В левом верхнем углу меняется название и цвет сезона
8	Довести хищника до голода	Появляется событие смерти в логге
9	Дождаться размножения животного	Появляется событие рождения в логге
10	Нажать CLEAR в логге	Лог очищается

Таблица 2. Результаты тестирования

№	Результат
1	Выполнено успешно
2	Выполнено успешно
3	Выполнено успешно
4	Выполнено успешно
5	Выполнено успешно
6	Выполнено успешно
7	Выполнено успешно
8	Выполнено успешно
9	Выполнено успешно
10	Выполнено успешно

Приложение В. Исходный код программы

ControlPanel.tsx:

```
/**
 * ControlPanel.tsx
 * Компонент панели управления симуляцией
 * Предоставляет три кнопки: Start, Pause, Reset
 */

// Интерфейс пропсов компонента
type Props = {
  isRunning: boolean;    // Флаг: запущена ли симуляция (влияет на активность кнопок)
  onStart: () => void;    // Callback для запуска симуляции
  onPause: () => void;    // Callback для паузы симуляции
  onReset: () => void;    // Callback для сброса симуляции
};

export default function ControlPanel({
  isRunning,
  onStart,
  onPause,
  onReset
}: Props) {

  // Базовые стили для всех кнопок (вынесены в объект для переиспользования)
  const btnBase = {
    padding: "14px 32px",    // Внутренние отступы
    borderRadius: "10px",    // Скругление углов
    border: "none",          // Убираем стандартную рамку
```

```

cursor: "pointer",      // Курсор-указатель при наведении
fontSize: "18px",      // Размер шрифта
fontWeight: "bold",    // Жирное начертание
transition: "all 0.2s", // Плавная анимация при наведении
minWidth: "130px",     // Минимальная ширина кнопки
};

return (
  // Контейнер кнопок: горизонтальное расположение с отступом между кнопками
  <div style={{ display: "flex", gap: "16px", marginTop: "8px" }}>

    {/* КНОПКА START */}
    <button
      style={{
        ...btnBase,
        // Цвет: зеленый, если симуляция не запущена; темно-зеленый, если запущена
        background: isRunning ? "#166534" : "#22c55e",
        // Прозрачность: 0.6 если кнопка неактивна, иначе 1
        opacity: isRunning ? 0.6 : 1,
        // Курсор: not-allowed если кнопка неактивна
        cursor: isRunning ? "not-allowed" : "pointer",
        // Тень для эффекта свечения
        boxShadow: "0 4px 12px rgba(34,197,94,0.3)",
      }}
      onClick={onStart} // Обработчик клика
      disabled={isRunning} // Неактивна, когда симуляция уже запущена
    >
      ► START

```

```
</button>
```

```
{/* КНОПКА PAUSE */}
```

```
<button
```

```
  style={{
```

```
    ...btnBase,
```

```
    // Цвет: оранжевый, если симуляция запущена; темно-оранжевый, если на паузе
```

```
    background: !isRunning ? "#92400e" : "#f59e0b",
```

```
    // Прозрачность: 0.6 если кнопка неактивна
```

```
    opacity: !isRunning ? 0.6 : 1,
```

```
    // Курсор: not-allowed если не на что ставить на паузу
```

```
    cursor: !isRunning ? "not-allowed" : "pointer",
```

```
    boxShadow: "0 4px 12px rgba(245,158,11,0.3)",
```

```
  }}  
  onClick={onPause}    // Обработчик клика
```

```
  disabled={!isRunning} // Неактивна, когда симуляция на паузе или не запущена
```

```
>
```

```
  II PAUSE
```

```
</button>
```

```
{/* КНОПКА RESET */}
```

```
<button
```

```
  style={{
```

```
    ...btnBase,
```

```
    background: "#dc2626",    // Красный цвет
```

```
    boxShadow: "0 4px 12px rgba(220,38,38,0.3)",
```

```
  }}
```

```
  onClick={onReset}    // Обработчик клика (всегда активна)
```


>

 RESET

</button>

</div>

);

}

EventLogPanel.tsx:

/**

* EventLogPanel.tsx

* Компонент панели лога событий

* Отображает все события симуляции (рождения, смерти, поедания, смены сезонов)

* Использует паттерн Singleton для получения экземпляра EventLog

*/

import { useEffect, useState } from "react";

import { EventLog } from "../utils/EventLog";

import type { LogEvent } from "../utils/EventLog";

export default function EventLogPanel() {

// Состояние: массив событий для отображения

const [events, setEvents] = useState<LogEvent[]>([]);

/**

* Подписка на обновления лога при монтировании компонента

* При каждом новом событии состояние обновляется автоматически

*/

```

useEffect(() => {

  const log = EventLog.getInstance();      // Получаем экземпляр Singleton
  setEvents(log.getEvents());              // Загружаем существующие события

  // Подписываемся на будущие обновления

  const unsubscribe = log.subscribe(() => {
    setEvents(log.getEvents());            // Обновляем состояние при новом событии
  });

  // Отписываемся при размонтировании компонента
  return () => unsubscribe();
}, []);

/**
 * Возвращает стили в зависимости от типа события
 * Цветовая кодировка:
 * - green: рождение
 * - red: смерть
 * - orange: поедание
 * - blue: смена сезона
 */
const getEventStyle = (type: LogEvent["type"]) => {
  switch (type) {
    case "birth":
      return { color: "#22c55e", borderLeft: "3px solid #22c55e" };
    case "death":
      return { color: "#ef4444", borderLeft: "3px solid #ef4444" };
    case "eat":

```

```

    return { color: "#f59e0b", borderLeft: "3px solid #f59e0b" };
case "season":
    return { color: "#60a5fa", borderLeft: "3px solid #60a5fa" };
default:
    return { color: "#94a3b8", borderLeft: "3px solid #94a3b8" };
}
};

```

```
/**
```

```
 * Форматирует timestamp в строку времени (ЧЧ:ММ:СС)
```

```
 */
```

```

const formatTime = (timestamp: number) => {
    const date = new Date(timestamp);
    return date.toLocaleTimeString();
};

```

```
return (
```

```
    // Основной контейнер панели лога
```

```
<div
```

```
    style={{
```

```
        background: "#1e293b",      // Темно-синий фон
```

```
        borderRadius: "16px",        // Скругление углов
```

```
        border: "1px solid #334155", // Тонкая рамка
```

```
        boxShadow: "0 8px 32px rgba(0,0,0,0.2)", // Тень
```

```
        width: "340px",              // Фиксированная ширина
```

```
        height: "350px",             // Фиксированная высота
```

```
        display: "flex",
```

```
        flexDirection: "column",

```

```

        overflow: "hidden",          // Скрываем 溢出
    }}
>
{ /* ЗАГОЛОВОК ПАНЕЛИ */ }
<div
    style={{
        padding: "16px 20px",
        borderBottom: "1px solid #334155", // Разделительная линия
        display: "flex",
        justifyContent: "space-between",
        alignItems: "center",
    }}
>
<h3 style={{ margin: 0, fontSize: "18px", color: "#fff" }}>
     Event Log
</h3>

{ /* КНОПКА ОЧИСТКИ ЛОГА */ }
<button
    onClick={() => EventLog.getInstance().clear()} // Очищаем лог через Singleton
    style={{
        background: "#334155",
        border: "none",
        borderRadius: "6px",
        padding: "4px 12px",
        color: "#fff",
        cursor: "pointer",
        fontSize: "12px",

```

```

    }}
  >
  Clear
</button>
</div>

{/* ОБЛАСТЬ СОБЫТИЙ (прокручиваемая) */}
<div
  style={{
    flex: 1,           // Занимает все свободное место
    overflowY: "auto",  // Вертикальная прокрутка при необходимости
    padding: "12px",
    fontSize: "13px",
  }}
>

  {/* Если событий нет - показываем сообщение */}
  {events.length === 0 ? (
    <div style={{ textAlign: "center", color: "#64748b", padding: "20px" }}>
      No events yet. Start simulation!
    </div>
  ) : (
    // Отображение списка событий
    events.map((event) => (
      <div
        key={event.id}           // Уникальный ключ для React
        style={{
          ...getEventStyle(event.type), // Стили в зависимости от типа
          padding: "8px 12px",

```

```

marginBottom: "8px",
background: "#0f172a",    // Темный фон для каждого события
borderRadius: "8px",
fontSize: "12px",
}}
>
    {/* Время события */}
    <div style={{ color: "#64748b", fontSize: "10px", marginBottom: "4px" }}>
        {formatTime(event.timestamp)}
    </div>
    {/* Текст события */}
    <div style={{ color: "#e2e8f0" }}>{event.message}</div>
</div>
))
)}
</div>
</div>
);
}

```

ObjectInfo.tsx:

```

/**
 * ObjectInfo.tsx
 * Компонент всплывающего информационного окна
 * Отображается при клике на животное (Prey или Predator)
 * Показывает подробную информацию: ID, тип, энергию, возраст, стадию, скорость, размер,
  статус
 */

```

```
// Интерфейс информационного объекта, получаемого от Animal.getInfo()
```

```
type ObjectInfo = {  
  id: number;      // Уникальный идентификатор животного  
  type: string;    // Тип с эмодзи (например, "🐾 Prey")  
  energy: number;  // Текущий уровень энергии  
  age: number;     // Возраст в тиках симуляции  
  ageYears: number; // Возраст в условных годах (age / 1000)  
  stage: string;   // Стадия жизни (Young / Adult / Old)  
  status: string;  // Статус (alive / dead)  
  speed: string;   // Текущая скорость передвижения  
  size: number;    // Размер для отрисовки  
};
```

```
// Пропсы компонента
```

```
type Props = {  
  object: ObjectInfo | null; // Объект для отображения (null = окно скрыто)  
  onClose: () => void;       // Callback для закрытия окна  
};
```

```
export default function ObjectInfo({ object, onClose }: Props) {
```

```
  // Если объект не выбран - ничего не отображаем
```

```
  if (!object) return null;
```

```
  /**
```

```
   * Возвращает цвет в зависимости от стадии жизни
```

```
   * Young -> зеленый, Adult -> оранжевый, Old -> красный
```

```
  */
```

```
const getStageColor = (stage: string) => {
  if (stage === "Young") return "#22c55e";
  if (stage === "Adult") return "#f59e0b";
  return "#ef4444";
};
```

```
/**
```

```
 * Возвращает иконку эмодзи в зависимости от стадии жизни
```

```
 * Young -> птенец, Adult -> олень, Old -> старик
```

```
 */
```

```
const getStageIcon = (stage: string) => {
  if (stage === "Young") return "🐣";
  if (stage === "Adult") return "🦌";
  return "👴";
};
```

```
/**
```

```
 * Возвращает цвет индикатора энергии в зависимости от уровня
```

```
 * >70 -> зеленый (высокий), >30 -> оранжевый (средний), <=30 -> красный (низкий)
```

```
 */
```

```
const getEnergyColor = (energy: number) => {
  if (energy > 70) return "#22c55e";
  if (energy > 30) return "#f59e0b";
  return "#ef4444";
};
```

```
return (
```

```
  // Всплывающее окно - фиксированная позиция в левом нижнем углу
```



```

<div
  style={{
    position: "fixed",    // Фиксированное позиционирование относительно окна браузера
    bottom: "20px",      // Отступ снизу 20 пикселей
    left: "20px",        // Отступ слева 20 пикселей (чтобы не перекрывать лог справа)
    background: "#1e293b", // Темно-синий фон
    borderRadius: "12px", // Скругление углов
    padding: "16px 20px", // Внутренние отступы
    border: "1px solid #334155", // Тонкая рамка
    boxShadow: "0 8px 32px rgba(0,0,0,0.4)", // Тень для объема
    zIndex: 1000,        // Высокий z-index, чтобы окно было поверх других элементов
    minWidth: "260px",   // Минимальная ширина окна
  }}
>

  { /* ЗАГОЛОВОК: тип и ID объекта + кнопка закрытия */ }

  <div style={{ display: "flex", justifyContent: "space-between", alignItems: "center", marginBottom:
"12px" }}>

    <h3 style={{ margin: 0, color: "#fff" }}>

      {object.type} #{object.id}

    </h3>

    <button

      onClick={onClose}

      style={{

        background: "none",

        border: "none",

        color: "#94a3b8",

        fontSize: "20px",

        cursor: "pointer",

```

```

    }}
  >
    ✕
  </button>
</div>

```

```

{ /* ОСНОВНАЯ ИНФОРМАЦИЯ */ }

```

```

<div style={{ fontSize: "14px", color: "#cbd5e1" }}>

```

```

{ /* Энергия с цветовым индикатором */ }

```

```

<div style={{ display: "flex", justifyContent: "space-between", marginBottom: "8px" }}>

```

```

  <span>♥ Energy:</span>

```

```

  <span style={{ color: getEnergyColor(object.energy), fontWeight:
"bold" }}>{object.energy}</span>

```

```

</div>

```

```

{ /* Возраст: в годах и тиках */ }

```

```

<div style={{ display: "flex", justifyContent: "space-between", marginBottom: "8px" }}>

```

```

  <span>📅 Age:</span>

```

```

  <span>{object.ageYears} years ({object.age} ticks)</span>

```

```

</div>

```

```

{ /* Стадия жизни с иконкой и цветом */ }

```

```

<div style={{ display: "flex", justifyContent: "space-between", marginBottom: "8px" }}>

```

```

  <span>★ Stage:</span>

```

```

  <span style={{ color: getStageColor(object.stage), fontWeight: "bold" }}>

```

```

    {getStageIcon(object.stage)} {object.stage}

```

```

  </span>

```

```
</div>
```

```
{/* Текущая скорость */}
```

```
<div style={{ display: "flex", justifyContent: "space-between", marginBottom: "8px" }}>
```

```
<span>⚡ Speed:</span>
```

```
<span>{object.speed}</span>
```

```
</div>
```

```
{/* Размер объекта на поле */}
```

```
<div style={{ display: "flex", justifyContent: "space-between", marginBottom: "8px" }}>
```

```
<span>📏 Size:</span>
```

```
<span>{object.size}px</span>
```

```
</div>
```

```
{/* Статус: жив или мертв */}
```

```
<div style={{ display: "flex", justifyContent: "space-between" }}>
```

```
<span>💀 Status:</span>
```

```
<span style={{ color: object.status === "alive" ? "#22c55e" : "#ef4444" }}>
```

```
{object.status === "alive" ? "Alive" : "Dead"}
```

```
</span>
```

```
</div>
```

```
</div>
```

```
</div>
```

```
);
```

```
}
```

SettingsPanel.tsx:

```
/**
 * SettingsPanel.tsx
 * Компонент панели настроек симуляции
 * Предоставляет ползунки для регулировки:
 * - количества травоядных (10-200)
 * - количества хищников (1-50)
 * - скорости травоядных (0.5-5)
 * - скорости хищников (0.5-6)
 */

// Интерфейс пропсов компонента
type Props = {
  // Текущие значения параметров
  preyCount: number;    // Количество травоядных
  predatorCount: number; // Количество хищников
  preySpeed: number;    // Скорость травоядных
  predatorSpeed: number; // Скорость хищников

  // Callback-функции для изменения параметров
  setPreyCount: (v: number) => void;
  setPredatorCount: (v: number) => void;
  setPreySpeed: (v: number) => void;
  setPredatorSpeed: (v: number) => void;
};

export default function SettingsPanel(props: Props) {
```

// Стили основной панели

```
const panel = {  
  background: "#1e293b",    // Темно-синий фон  
  padding: "28px 24px",     // Внутренние отступы  
  borderRadius: "20px",     // Скругление углов  
  border: "1px solid #334155", // Тонкая рамка  
  boxShadow: "0 8px 32px rgba(0,0,0,0.3)", // Тень  
  width: "320px",           // Фиксированная ширина  
};
```

// Стили для блока каждого параметра

```
const block = {  
  marginBottom: "28px",     // Отступ снизу между параметрами  
};
```

// Стили для метки (название параметра + значение)

```
const label = {  
  fontSize: "18px",         // Размер шрифта  
  fontWeight: "600" as const, // Полужирное начертание  
  color: "#e2e8f0",         // Светло-серый цвет  
  marginBottom: "12px",     // Отступ снизу до ползунка  
  display: "flex",          // Flex для расположения в строку  
  justifyContent: "space-between", // Название слева, значение справа  
};
```

// Стили для отображения числового значения параметра

```
const value = {  
  color: "#22c55e",         // Зеленый цвет
```

```

fontWeight: "bold",      // Жирное начертание
fontSize: "20px",        // Крупный шрифт
};

// Базовые стили для ползунка (input range)
const slider = {
  width: "100%",          // На всю ширину контейнера
  height: "8px",          // Толщина полосы
  borderRadius: "4px",    // Скругление
  background: "#334155",  // Темно-серая полоска
  outline: "none",        // Убираем обводку при фокусе
  WebkitAppearance: "none" as const, // Отключаем стандартный стиль Webkit
};

return (
  <div style={panel}>

    {/* ЗАГОЛОВОК ПАНЕЛИ */}
    <h3 style={{ margin: "0 0 28px 0", fontSize: "24px", color: "#fff", textAlign: "center" }}>
      ⚙ Settings
    </h3>

    {/* НАСТРОЙКА: Количество травоядных */}
    <div style={block}>
      <label style={label}>
        <span>🐇 Preys</span>
        <span style={value}>{props.preyCount}</span>
      </label>

```

```

<input
  style={slider}
  type="range"          // Тип "ползунок"
  min={10}              // Минимальное значение
  max={200}             // Максимальное значение
  value={props.preyCount} // Текущее значение
  onChange={(e) => props.setPreyCount(+e.target.value)} // Обновление состояния
/>
</div>

```

```

{/* НАСТРОЙКА: Количество хищников */}

```

```

<div style={block}>
  <label style={label}>
    <span>🐾 Predators</span>
    <span style={value}>{props.predatorCount}</span>
  </label>
  <input
    style={slider}
    type="range"
    min={1}
    max={50}
    value={props.predatorCount}
    onChange={(e) => props.setPredatorCount(+e.target.value)}
  />
</div>

```

```

{/* НАСТРОЙКА: Скорость травоядных */}

```

```

<div style={block}>

```

```

<label style={label}>
  <span>🐾 Prey Speed</span>
  <span style={value}>{props.preySpeed.toFixed(1)}</span> { /* 1 знак после запятой */ }
</label>
<input
  style={slider}
  type="range"
  min={0.5}
  max={5}
  step={0.1} // Шаг изменения 0.1
  value={props.preySpeed}
  onChange={(e) => props.setPreySpeed(+e.target.value)}
/>
</div>

```

```

{ /* НАСТРОЙКА: Скорость хищников */ }
<div style={block}>
  <label style={label}>
    <span>🐾 Predator Speed</span>
    <span style={value}>{props.predatorSpeed.toFixed(1)}</span>
  </label>
  <input
    style={slider}
    type="range"
    min={0.5}
    max={6}
    step={0.1}
    value={props.predatorSpeed}

```



```

    onChange={(e) => props.setPredatorSpeed(+e.target.value)}

  />

</div>

</div>

);
}

```

SimulationCanvas.tsx:

```

/**
 * SimulationCanvas.tsx
 * Компонент холста для отрисовки симуляции
 * Отвечает за:
 * - отрисовку всех объектов (трава, травоядные, хищники)
 * - отображение сезонных эффектов
 * - обработку кликов мыши для выбора объектов
 * - отображение информационного окна при клике на животное
 */

import { useEffect, useRef, useState } from "react";
import { World } from "../models/World";
import ObjectInfo from "../ObjectInfo";

// Пропсы компонента
type Props = {
  world: World; // Объект мира со всеми данными
  onObjectClick?: (info: any) => void; // Опциональный callback при клике на объект
};

```

```

export default function SimulationCanvas({ world, onObjectClick }: Props) {

  // Ссылка на DOM-элемент canvas

  const canvasRef = useRef<HTMLCanvasElement>(null);

  // Состояние: выбранный объект для отображения информации

  const [selectedObject, setSelectedObject] = useState<any>(null);

  /**
   * Обработчик клика по canvas
   * Определяет, по какому объекту кликнули, и показывает информацию о нем
   */

  const handleCanvasClick = (e: React.MouseEvent<HTMLCanvasElement>) => {

    const canvas = canvasRef.current;

    if (!canvas) return;

    // Получаем координаты клика относительно canvas с учетом масштабирования

    const rect = canvas.getBoundingClientRect();

    const scaleX = canvas.width / rect.width;

    const scaleY = canvas.height / rect.height;

    const mouseX = (e.clientX - rect.left) * scaleX;

    const mouseY = (e.clientY - rect.top) * scaleY;

    // Проверяем клик по травоядным

    for (const prey of world.preys) {

      const dx = prey.x - mouseX;

      const dy = prey.y - mouseY;
    }
  }
}

```

```

const distance = Math.hypot(dx, dy);

// Если клик в пределах радиуса объекта + 2 пикселя
if (distance < prey.getSize() + 2 && prey.alive) {
    setSelectedObject(prey.getInfo());
    if (onObjectClick) onObjectClick(prey.getInfo());
    return;
}

// Проверяем клик по хищникам
for (const predator of world.predators) {
    const dx = predator.x - mouseX;
    const dy = predator.y - mouseY;
    const distance = Math.hypot(dx, dy);
    if (distance < predator.getSize() + 2 && predator.alive) {
        setSelectedObject(predator.getInfo());
        if (onObjectClick) onObjectClick(predator.getInfo());
        return;
    }
}

// Если кликнули мимо объектов - закрываем информационное окно
setSelectedObject(null);
};

/**
 * Эффект для анимации и отрисовки
 * Запускает бесконечный цикл requestAnimationFrame

```

```

*/

useEffect(() => {

  const canvas = canvasRef.current;

  if (!canvas) return;

  const ctx = canvas.getContext("2d");

  if (!ctx) return;

  /**

  * Основная функция отрисовки
  * Вызывается на каждом кадре анимации
  */

  const draw = () => {

    // Очищаем canvas

    ctx.clearRect(0, 0, canvas.width, canvas.height);

    // Рисуем все слои

    drawGrass(ctx);      // Трава
    drawPreys(ctx);      // Травоядные
    drawPredators(ctx);  // Хищники
    drawSeasonIndicator(ctx); // Индикатор сезона

    // Запрашиваем следующий кадр

    requestAnimationFrame(draw);

  };

  draw();

  /**

```

* Отрисовка травы

* Трава отображается в виде зеленых кружков радиусом 3 пикселя

*/

```
function drawGrass(ctx: CanvasRenderingContext2D) {  
  for (const grass of world.grasses) {  
    if (!grass.isAvailable) continue; // Съеденная трава не отображается  
    ctx.beginPath();  
    ctx.fillStyle = "#22c55e";  
    ctx.arc(grass.x, grass.y, 3, 0, Math.PI * 2);  
    ctx.fill();  
  }  
}
```

/**

* Отрисовка травоядных

* Цвет зависит от стадии и энергии

* Размер зависит от стадии

* Отображается ID и индикатор энергии

*/

```
function drawPreys(ctx: CanvasRenderingContext2D) {  
  for (const prey of world.preys) {  
    if (!prey.alive) continue;  
    ctx.beginPath();  
  
    // Цвет от стадии и энергии  
    ctx.fillStyle = prey.getColor();  
    const size = prey.getSize();  
    ctx.arc(pre.x, pre.y, size, 0, Math.PI * 2);
```

```

ctx.fill();

// Обводка для разных стадий
ctx.strokeStyle = prey.getStage() === "Young" ? "#60a5fa" :
    prey.getStage() === "Adult" ? "#2563eb" : "#1e3a8a";
ctx.lineWidth = 1.5;
ctx.stroke();

// ID объекта
ctx.fillStyle = "#e2e8f0";
ctx.font = "bold 10px Arial";
ctx.shadowBlur = 0;
ctx.fillText(`${prey.id}`, prey.x - 6, prey.y - 8);

// Индикатор энергии (зеленая полоска под объектом)
const energyPercent = prey.energy / 100;
ctx.fillStyle = `rgba(34, 197, 94, ${energyPercent})`;
ctx.fillRect(prex.x - size, prey.y - size - 4, size * 2 * energyPercent, 3);
}
}

/**
 * Отрисовка хищников
 * Цвет зависит от стадии и энергии
 * Размер зависит от стадии
 * Отображается ID и индикатор энергии
 */
function drawPredators(ctx: CanvasRenderingContext2D) {

```

```

for (const predator of world.predators) {
  if (!predator.alive) continue;
  ctx.beginPath();

  ctx.fillStyle = predator.getColor();
  const size = predator.getSize();
  ctx.arc(predator.x, predator.y, size, 0, Math.PI * 2);
  ctx.fill();

  // Обводка для разных стадий
  ctx.strokeStyle = predator.getStage() === "Young" ? "#f87171" :
    predator.getStage() === "Adult" ? "#dc2626" : "#991b1b";
  ctx.lineWidth = 1.5;
  ctx.stroke();

  // ID объекта
  ctx.fillStyle = "#e2e8f0";
  ctx.font = "bold 10px Arial";
  ctx.fillText(`${predator.id}`, predator.x - 6, predator.y - 10);

  // Индикатор энергии (красная полоска под объектом)
  const energyPercent = predator.energy / 140;
  ctx.fillStyle = `rgba(239, 68, 68, ${energyPercent})`;
  ctx.fillRect(predator.x - size, predator.y - size - 4, size * 2 * energyPercent, 3);
}
}

/**

```

- * Отрисовка индикатора текущего сезона
- * Отображает название сезона с цветом и декоративными эффектами
- */

```
function drawSeasonIndicator(ctx: CanvasRenderingContext2D) {

  const seasonName = world.getSeasonName();

  const seasonColor = world.getSeasonColor();

  // Название сезона в левом верхнем углу

  ctx.fillStyle = seasonColor;

  ctx.font = "bold 18px Arial";

  ctx.shadowBlur = 0;

  ctx.fillText(seasonName, 15, 35);

  // Декоративные эффекты в зависимости от сезона

  ctx.font = "14px Arial";

  if (seasonName.includes("Winter")) {

    // Снежинки зимой

    ctx.fillStyle = "rgba(255,255,255,0.15)";

    for (let i = 0; i < 15; i++) {

      ctx.fillText("❄", (Math.random() * 800), (Math.random() * 600));

    }

  } else if (seasonName.includes("Spring")) {

    // Цветы весной

    ctx.fillStyle = "rgba(34,197,94,0.1)";

    for (let i = 0; i < 10; i++) {

      ctx.fillText("🌸", (Math.random() * 800), (Math.random() * 600));

    }

  } else if (seasonName.includes("Autumn")) {
```



```

// Листья осенью

ctx.fillStyle = "rgba(249,115,22,0.1)";

for (let i = 0; i < 10; i++) {

  ctx.fillText("🍁", (Math.random() * 800), (Math.random() * 600));

}

}

}, [world]); // Перерисовка при изменении мира

return (
  <
  { /* Canvas для отрисовки */ }
  <canvas
    ref={canvasRef}
    width={800}
    height={600}
    onClick={handleCanvasClick}
    style={{
      borderRadius: "16px",
      background: "#0b1220",
      boxShadow: "0 8px 32px rgba(0,0,0,0.4)",
      border: "2px solid #1e293b",
      display: "block",
      cursor: "pointer",
    }}
  />

  { /* Информационное окно при выбранном объекте */ }
  <ObjectInfo object={selectedObject} onClose={() => setSelectedObject(null)} />

```

```

</>

);
}

```

StatisticsPanel.tsx:

```

/**
 * StatisticsPanel.tsx
 * Компонент панели статистики
 * Отображает ключевые показатели симуляции:
 * - текущий сезон года
 * - время симуляции
 * - количество травоядных
 * - количество хищников
 */

import { World } from "../models/World";

// Пропсы компонента
type Props = {
  world: World; // Объект мира для получения статистических данных
};

export default function StatisticsPanel({ world }: Props) {

  // Стили для карточки статистики
  const card = {
    background: "#1e293b", // Темно-синий фон
    padding: "20px 24px", // Внутренние отступы

```

```
borderRadius: "16px",      // Скругление углов
marginBottom: "16px",     // Отступ между карточками
textAlign: "center" as const, // Выравнивание по центру
border: "1px solid #334155", // Тонкая рамка
boxShadow: "0 4px 12px rgba(0,0,0,0.15)", // Легкая тень
minWidth: "200px",       // Минимальная ширина
};
```

// Стили для подписи (названия показателя)

```
const label = {
  fontSize: "15px",      // Размер шрифта
  color: "#94a3b8",      // Серый цвет
  marginBottom: "10px",  // Отступ снизу до значения
  letterSpacing: "1px",  // Межбуквенный интервал
  fontWeight: "500" as const, // Средняя жирность
};
```

// Стили для числового значения (время, количество)

```
const value = {
  fontSize: "42px",      // Крупный шрифт
  fontWeight: "bold" as const, // Жирное начертание
  margin: 0,             // Убираем отступы
  // Градиентный текст от белого к серому
  background: "linear-gradient(135deg, #fff, #94a3b8)",
  backgroundClip: "text",
  WebkitBackgroundClip: "text",
  color: "transparent",  // Прозрачный цвет для градиента
};
```

```

// Стили для отображения сезона (с динамическим цветом)

const seasonValue = {

  fontSize: "24px",          // Размер шрифта

  fontWeight: "bold" as const, // Жирное начертание

  margin: 0,

  color: world.getSeasonColor(), // Цвет берется из SeasonManager

};

return (

  <div>

    { /* КАРТОЧКА СЕЗОНА */ }

    <div style={card}>

      <div style={label}><img alt="Season icon" data-bbox="300 500 320 515"/> SEASON</div>

      <div style={seasonValue}>{world.getSeasonName()}</div>

      { /* Пример вывода: "🌸 Spring", "☀️ Summer", "🍂 Autumn", "❄️ Winter" */ }

    </div>

    { /* КАРТОЧКА ВРЕМЕНИ */ }

    <div style={card}>

      <div style={label}><img alt="Time icon" data-bbox="300 720 320 735"/> TIME</div>

      <h2 style={value}>{world.time.toFixed(1)}</h2>

      { /* Время в условных единицах, 1 знак после запятой */ }

    </div>

    { /* КАРТОЧКА КОЛИЧЕСТВА ТРАВЯДНЫХ */ }

    <div style={card}>

```

```

<div style={label}>🐇 PREYS</div>

<h2 style={value}>{world.preys.length}</h2>

{/* Текущее количество живых травоядных */}

</div>

```

```

{/* КАРТОЧКА КОЛИЧЕСТВА ХИЩНИКОВ */}

<div style={card}>

  <div style={label}>🐺 PREDATORS</div>

  <h2 style={value}>{world.predators.length}</h2>

  {/* Текущее количество живых хищников */}

</div>

```

```

</div>

```

```

);

```

```

}

```

useSimulation.ts:

```

/**

```

```

 * useSimulation.ts

```

```

 * Кастомный хук для управления симуляцией

```

```

 * Отвечает за:

```

```

 * - создание и хранение экземпляра World

```

```

 * - запуск/остановку цикла обновления

```

```

 * - сброс симуляции с новыми параметрами

```

```

 */

```

```

import { useEffect, useRef, useState } from "react";

```

```

import { World } from "../models/World";

```

```

/**
 * Хук управления симуляцией
 * @param preyCount - начальное количество травоядных
 * @param predatorCount - начальное количество хищников
 * @param preySpeed - базовая скорость травоядных
 * @param predatorSpeed - базовая скорость хищников
 * @returns объект с данными и методами управления
 */
export function useSimulation(
  preyCount: number,
  predatorCount: number,
  preySpeed: number,
  predatorSpeed: number
) {
  // useRef для хранения экземпляра World (не вызывает перерисовку при изменении)
  const worldRef = useRef<World | null>(null);

  // Ленивая инициализация: создаем World только один раз при первом рендере
  if (!worldRef.current) {
    worldRef.current = new World(
      800,          // Ширина поля
      600,          // Высота поля
      preyCount,    // Количество травоядных
      predatorCount, // Количество хищников
      preySpeed,    // Скорость травоядных
      predatorSpeed // Скорость хищников
    );
  }
}

```

```

}

// Состояние: запущена ли симуляция
const [isRunning, setIsRunning] = useState(false);

// Состояние для принудительной перерисовки компонентов
// Значение не используется, важна только смена состояния
const [, forceUpdate] = useState(0);

/**
 * Эффект для запуска цикла обновления симуляции
 * Срабатывает при изменении isRunning
 */
useEffect(() => {
  // Если симуляция не запущена или мир не создан - ничего не делаем
  if (!isRunning || !worldRef.current) return;

  // Запускаем интервал обновления (30 мс = ~33 кадра в секунду)
  const interval = setInterval(() => {
    if (!worldRef.current) return;

    const world = worldRef.current;

    // Выполняем один тик симуляции
    world.update();

    // Автоматическая остановка, если одна из популяций вымерла
    if (world.preys.length === 0 || world.predators.length === 0) {

```

```

    setIsRunning(false);

    return;
}

// Принудительная перерисовка UI (обновление статистики, canvas и т.д.)
forceUpdate(v => v + 1);
}, 30); // 30 миллисекунд между тиками

// Очистка интервала при размонтировании или остановке
return () => clearInterval(interval);
}, [isRunning]); // Зависимость: пересоздаем интервал при изменении isRunning

/**
 * Запуск симуляции
 */
const start = () => setIsRunning(true);

/**
 * Пауза симуляции
 */
const pause = () => setIsRunning(false);

/**
 * Сброс симуляции
 * Создает новый мир с текущими настройками параметров
 */
const reset = () => {
    // Создаем новый экземпляр World с актуальными параметрами

```



```

worldRef.current = new World(
  800,
  600,
  preyCount,    // Текущее значение из состояния App
  predatorCount, // Текущее значение из состояния App
  preySpeed,    // Текущее значение из состояния App
  predatorSpeed // Текущее значение из состояния App
);
// Вызываем принудительную перерисовку
forceUpdate((v) => v + 1);
};

// Возвращаем данные и методы для использования в компонентах
return {
  world: worldRef.current, // Экземпляр мира (всегда актуальный)
  isRunning,               // Флаг запуска
  start,                   // Функция запуска
  pause,                   // Функция паузы
  reset,                   // Функция сброса
};
}

```

Animal.ts:

```

/**
 * Animal.ts
 * Абстрактный базовый класс для всех животных в симуляции
 *

```

* Реализует общую логику:

* - уникальные ID для каждого животного

* - движение (к цели или случайное)

* - старение и потерю энергии

* - стадии жизни (Young/Adult/Old)

* - смерть от голода или старости

* - размножение

*

* Дочерние классы (Prey, Predator) реализуют специфическое поведение:

* - поиск цели (трава или добыча)

* - получение энергии от еды

* - параметры стадий

*/

```
import { EventLog } from "../utils/EventLog";
```

```
export abstract class Animal {
```

```
  // Основные характеристики животного
```

```
  id: number;      // Уникальный идентификатор
```

```
  x: number;      // Координата X на поле (0-800)
```

```
  y: number;      // Координата Y на поле (0-600)
```

```
  energy: number;  // Уровень энергии (влияет на выживание и размножение)
```

```
  age: number = 0; // Возраст в тиках (увеличивается каждый update)
```

```
  speed: number;   // Базовая скорость передвижения
```

```
  alive: boolean = true; // Статус жизни
```

```
  // Статический счетчик для генерации уникальных ID (общий для всех животных)
```

```
  protected static nextId: number = 1;
```

```

/**
 * Конструктор базового класса
 * @param x - начальная координата X
 * @param y - начальная координата Y
 * @param speed - базовая скорость
 * @param initialEnergy - начальная энергия
 */
constructor(x: number, y: number, speed: number, initialEnergy: number) {
  // Генерация уникального ID через статический счетчик
  const constructor = this.constructor as typeof Animal;
  this.id = constructor.nextId;
  constructor.nextId++;

  // Инициализация координат и параметров
  this.x = x;
  this.y = y;
  this.speed = speed;
  this.energy = initialEnergy;

  // Логируем рождение животного
  const log = EventLog.getInstance();
  log.addEvent(`${this.getEmoji()} ${this.getTypeName()} #${this.id} was born at position
    (${Math.floor(x)}, ${Math.floor(y)})`, "birth");
}

// ===== АБСТРАКТНЫЕ МЕТОДЫ (реализуются в Prey и Predator) =====

```

```

abstract getTypeName(): string;      // Возвращает "Prey" или "Predator"
abstract getEmoji(): string;        // Возвращает "🐾" или "🦋"
abstract getMaxAge(): number;        // Максимальный возраст (5000 или 6000)
abstract getReproductionEnergy(): number; // Порог энергии для размножения (100 или 140)
abstract getInitialEnergy(): number;  // Начальная энергия (50 или 70)
abstract getEnergyDecayRate(): number; // Потеря энергии за тик (0.05 или 0.08)
abstract getReproductionEnergyCost(): number; // Затраты энергии на размножение (40 или 50)
abstract getEatEnergyGain(): number;   // Энергия от еды (20 или 40)
abstract getStageBoundaries(): { young: number; adult: number }; // Границы стадий
abstract getSpeedMultipliers(): { young: number; adult: number; old: number }; // Множители
// скорости
abstract getSizeMultipliers(): { young: number; adult: number; old: number }; // Множители размера

// ===== МЕТОДЫ СТАДИЙ ЖИЗНИ =====

/**
 * Определяет текущую стадию жизни по возрасту
 * Young -> молодой (быстрый, маленький)
 * Adult -> взрослый (нормальный, может размножаться)
 * Old -> старый (медленный, темный)
 */
getStage(): string {
    const boundaries = this.getStageBoundaries();
    if (this.age < boundaries.young) return "Young";
    if (this.age < boundaries.adult) return "Adult";
    return "Old";
}

```

```

/**
 * Возвращает текущую скорость с учетом стадии жизни
 */
getCurrentSpeed(): number {
    const stage = this.getStage();
    const multipliers = this.getSpeedMultipliers();
    if (stage === "Young") return this.speed * multipliers.young;
    if (stage === "Old") return this.speed * multipliers.old;
    return this.speed * multipliers.adult;
}

```

```

/**
 * Возвращает размер для отрисовки с учетом стадии жизни
 */
getSize(): number {
    const stage = this.getStage();
    const multipliers = this.getSizeMultipliers();
    const baseSize = this.getTypeName() === "Prey" ? 5 : 7;
    if (stage === "Young") return baseSize * multipliers.young;
    if (stage === "Adult") return (baseSize + 2) * multipliers.adult;
    return (baseSize + 1) * multipliers.old;
}

```

```

// ===== ИНФОРМАЦИОННЫЕ МЕТОДЫ =====

```

```

/**
 * Возвращает полную информацию об объекте для UI
 */

```

```

getInfo() {
  const ageYears = Math.floor(this.age / 1000); // Перевод в условные годы
  return {
    id: this.id,
    type: `${this.getEmoji()} ${this.getTypeName()}`,
    energy: Math.floor(this.energy),
    age: this.age,
    ageYears: ageYears,
    stage: this.getStage(),
    status: this.alive ? "alive" : "dead",
    speed: this.getCurrentSpeed().toFixed(1),
    size: this.getSize(),
  };
}

// ===== МЕТОДЫ РАЗМНОЖЕНИЯ И СМЕРТИ =====

/**
 * Проверяет, может ли животное размножаться
 * Условия: достаточно энергии И стадия "Adult"
 */
canReproduce(): boolean {
  return this.energy >= this.getReproductionEnergy() && this.getStage() === "Adult";
}

/**
 * Базовое обновление: увеличение возраста, потеря энергии
 * Вызывается в update() дочерних классов

```

```

*/

updateBase(): void {
    if (!this.alive) return;

    this.age++;

    this.energy -= this.getEnergyDecayRate();
}

/**
 * Проверка условий смерти
 * - смерть от голода (energy <= 0)
 * - смерть от старости (age > MAX_AGE)
 */

checkDeath(): void {
    if (!this.alive) return;

    const log = EventLog.getInstance();

    if (this.energy <= 0) {
        log.addEvent(`💀 ${this.getEmoji()} ${this.getTypeName()} #${this.id} died from starvation at age
        ${Math.floor(this.age / 1000)} years`, "death");

        this.alive = false;
    } else if (this.age > this.getMaxAge() && this.alive) {
        log.addEvent(`👴 ${this.getEmoji()} ${this.getTypeName()} #${this.id} died from old age at
        ${Math.floor(this.age / 1000)} years`, "death");

        this.alive = false;
    }
}

// ===== МЕТОДЫ ДВИЖЕНИЯ =====

```

```
/**
```

```
 * Движение к целевой точке (трава или добыча)
```

```
 * Использует нормализованный вектор направления
```

```
 */
```

```
moveTowards(targetX: number, targetY: number, worldWidth: number, worldHeight: number) {
```

```
    const dx = targetX - this.x;
```

```
    const dy = targetY - this.y;
```

```
    const distance = Math.hypot(dx, dy);
```

```
    if (distance === 0) return;
```

```
    const speed = this.getCurrentSpeed();
```

```
    this.x += (dx / distance) * speed;
```

```
    this.y += (dy / distance) * speed;
```

```
    this.keepInsideWorld(worldWidth, worldHeight);
```

```
}
```

```
/**
```

```
 * Случайное движение (когда нет цели)
```

```
 */
```

```
moveRandom(worldWidth: number, worldHeight: number) {
```

```
    const speed = this.getCurrentSpeed();
```

```
    this.x += (Math.random() - 0.5) * speed * 2;
```

```
    this.y += (Math.random() - 0.5) * speed * 2;
```

```
    this.keepInsideWorld(worldWidth, worldHeight);
```

```
}
```

```
/**
```


* Удерживает объект в границах игрового поля

*/

```
keepInsideWorld(width: number, height: number) {  
  this.x = Math.max(0, Math.min(width, this.x));  
  this.y = Math.max(0, Math.min(height, this.y));  
}  
}
```

Grass.ts:

/**

* Grass.ts

* Класс, представляющий растение (траву) в симуляции

*

* Трава является источником пищи для травоядных (Prey)

* Особенности:

* - Съеденная трава восстанавливается через определенное время

* - Скорость восстановления зависит от текущего сезона

* - Трава отображается на поле в виде зеленых точек

*/

```
import { SeasonManager } from "../SeasonManager";
```

```
export class Grass {
```

```
  // Координаты травы на игровом поле
```

```
  x: number;
```

```
  y: number;
```

```
  // Доступность травы для поедания
```

```

// true - трава есть, false - съедена (восстанавливается)

isAvailable: boolean = true;

// Время восстановления травы в тиках (базовое значение 300 = ~9 секунд)

private regrowTime: number = 300;

// Таймер обратного отсчета до восстановления

private timer: number = 0;

// Ссылка на менеджер сезонов для получения коэффициента роста

private seasonManager: SeasonManager | null = null;

/**
 * Конструктор
 * @param x - координата X на поле (0-800)
 * @param y - координата Y на поле (0-600)
 */
constructor(x: number, y: number) {
    this.x = x;
    this.y = y;
}

/**
 * Устанавливает менеджер сезонов (вызывается из World)
 * @param manager - экземпляр SeasonManager
 */
setSeasonManager(manager: SeasonManager) {
    this.seasonManager = manager;
}

```

```
}
```

```
/**
```

```
* Метод поедания травы (вызывается травоядным)
```

```
* - Делает траву недоступной
```

```
* - Запускает таймер восстановления
```

```
* - Время восстановления зависит от сезона:
```

```
* * Весна (множитель 1.5) -> трава растет быстрее
```

```
* * Лето (множитель 1.2) -> нормальный рост
```

```
* * Осень (множитель 0.8) -> медленный рост
```

```
* * Зима (множитель 0.3) -> очень медленный рост
```

```
*/
```

```
eat() {
```

```
    this.isAvailable = false;
```

```
    // Получаем множитель роста от текущего сезона
```

```
    const multiplier = this.seasonManager?.getGrassGrowthMultiplier() || 1;
```

```
    // Время восстановления = базовое время / множитель
```

```
    this.timer = this.regrowTime / multiplier;
```

```
}
```

```
/**
```

```
* Обновление состояния травы (вызывается каждый тик из World)
```

```
* @param worldStopped - флаг, остановлена ли симуляция
```

```
*/
```

```
update(worldStopped: boolean = false) {
```

```
    // Если симуляция на паузе - ничего не обновляем
```

```
    if (worldStopped) return;
```

```
// Если трава доступна - не нужно обновлять таймер
```

```
if (this.isAvailable) return;
```

```
// Уменьшаем таймер восстановления
```

```
this.timer--;
```

```
// Когда таймер достиг нуля - трава снова доступна
```

```
if (this.timer <= 0) {
```

```
    this.isAvailable = true;
```

```
}
```

```
}
```

```
}
```

Predator.ts:

```
/**
```

```
 * Predator.ts
```

```
 * Класс хищника, наследуется от Animal
```

```
 *
```

```
 * Особенности поведения хищника:
```

```
 * - Активно охотится на травоядных (Prey)
```

```
 * - Двигается к ближайшей жертве
```

```
 * - При контакте съедает жертву и восполняет энергию
```

```
 * - Имеет свои параметры: максимальный возраст 6000, энергия размножения 140
```

```
 * - Цвет отображается в красных тонах (меняется от стадии и энергии)
```

```
 */
```

```
import { Prey } from "../Prey";
```

```
import { Animal } from "../Animal";
```

```

import { EventLog } from "../utils/EventLog";

export class Predator extends Animal {
    // Статические параметры для всех хищников
    static MAX_AGE = 6000;           // Максимальный возраст (тиков)
    static REPRODUCTION_ENERGY = 140; // Энергия, необходимая для размножения

    /**
     * Конструктор хищника
     * @param x - начальная координата X
     * @param y - начальная координата Y
     * @param speed - базовая скорость (по умолчанию 2)
     */
    constructor(x: number, y: number, speed: number = 2) {
        super(x, y, speed, 70); // Начальная энергия = 70
    }

    // ===== РЕАЛИЗАЦИЯ АБСТРАКТНЫХ МЕТОДОВ ANIMAL =====

    getTypeName(): string {
        return "Predator";
    }

    getEmoji(): string {
        return "🐾";
    }

    getMaxAge(): number {

```

```
return Predator.MAX_AGE;  
}
```

```
getReproductionEnergy(): number {  
    return Predator.REPRODUCTION_ENERGY;  
}
```

```
getInitialEnergy(): number {  
    return 70;  
}
```

```
getEnergyDecayRate(): number {  
    return 0.08;    // Потеря 0.08 энергии за тик (больше чем у Prey)  
}
```

```
getReproductionEnergyCost(): number {  
    return 50;    // Тратит 50 энергии на размножение  
}
```

```
getEatEnergyGain(): number {  
    return 40;    // Получает 40 энергии за съеденную жертву  
}
```

```
/**
```

```
* Границы стадий жизни для хищника
```

```
* Young: 0-1500 тиков
```

```
* Adult: 1500-4000 тиков
```

```
* Old: 4000+ тиков
```

*/

```
getStageBoundaries(): { young: number; adult: number } {  
    return { young: 1500, adult: 4000 };  
}
```

/**

* Множители скорости в зависимости от стадии

* Young: быстрее на 20%

* Adult: нормальная скорость

* Old: медленнее на 40%

*/

```
getSpeedMultipliers(): { young: number; adult: number; old: number } {  
    return { young: 1.2, adult: 1.0, old: 0.6 };  
}
```

/**

* Множители размера в зависимости от стадии

* Young: 90% от базового

* Adult: 130% от базового (самые крупные)

* Old: 110% от базового

*/

```
getSizeMultipliers(): { young: number; adult: number; old: number } {  
    return { young: 0.9, adult: 1.3, old: 1.1 };  
}
```

// ===== СПЕЦИФИЧЕСКИЕ МЕТОДЫ ХИЩНИКА =====

/**

- * Возвращает цвет хищника для отрисовки
- * Цвет зависит от стадии и уровня энергии
- * - Young: светлый красный (rgba(248, 113, 113))
- * - Adult: насыщенный красный (rgba(239, 68, 68))
- * - Old: темный красный (rgba(153, 27, 27))
- * Прозрачность увеличивается при высокой энергии
- */

```
getColor(): string {
    const stage = this.getStage();

    const energyPercent = this.energy / 140; // Максимальная энергия 140

    if (stage === "Young") {
        return `rgba(248, 113, 113, ${0.6 + energyPercent * 0.4})`;
    }

    if (stage === "Adult") {
        return `rgba(239, 68, 68, ${0.7 + energyPercent * 0.3})`;
    }

    return `rgba(153, 27, 27, ${0.4 + energyPercent * 0.3})`;
}
```

/**

- * Основной метод обновления состояния хищника
- * Вызывается каждый тик симуляции
- * @param preys - массив всех травоядных
- * @param worldWidth - ширина поля (для границ)
- * @param worldHeight - высота поля (для границ)
- */

```
update(preys: Prey[], worldWidth: number, worldHeight: number) {
```



```

if (!this.alive) return;

// Базовое обновление (возраст, энергия)
this.updateBase();

// Поиск ближайшей жертвы
const target = this.findNearestPrey(preys);

if (target) {
    // Движение к жертве
    this.moveTowards(target.x, target.y, worldWidth, worldHeight);

    // Проверка контакта с жертвой (дистанция < 8 пикселей)
    const distance = Math.hypot(target.x - this.x, target.y - this.y);
    if (distance < 8 && target.alive) {
        // Поедание жертвы
        const log = EventLog.getInstance();
        log.addEvent(`☠ Predator #${this.id} ate Prey #${target.id}! +${this.getEatEnergyGain()} energy`,
            "eat");

        target.alive = false; // Жертва умирает
        this.energy += this.getEatEnergyGain(); // Хищник получает энергию
        if (this.energy > 140) this.energy = 140; // Ограничение максимальной энергии
    }
}

// Проверка условий смерти (голод или старость)
this.checkDeath();

```

```

}

/**
 * Поиск ближайшего живого травоядного
 * Используется для навигации хищника
 * @param preys - массив всех травоядных
 * @returns ближайший объект Prey или null, если жертв нет
 */
private findNearestPrey(preys: Prey[]): Prey | null {
    // Фильтруем только живых травоядных
    const alivePreys = preys.filter(p => p.alive);
    if (alivePreys.length === 0) return null;

    // Поиск с минимальным евклидовым расстоянием
    let nearest = alivePreys[0];
    let minDistance = Infinity;

    for (const prey of alivePreys) {
        const distance = Math.hypot(prey.x - this.x, prey.y - this.y);
        if (distance < minDistance) {
            minDistance = distance;
            nearest = prey;
        }
    }

    return nearest;
}

/**

```

```

* Размножение хищника

* Создает нового хищника рядом с родителем

* @returns новый объект Predator

*/

reproduce(): Predator {

    // Тратим энергию на размножение

    this.energy -= this.getReproductionEnergyCost();


    // Логируем событие

    const log = EventLog.getInstance();

    log.addEvent(`🐾 Predator #${this.id} reproduced! New Predator will be born`, "birth");


    // Создаем потомка со случайным смещением до 20 пикселей

    return new Predator(

        this.x + (Math.random() - 0.5) * 20,

        this.y + (Math.random() - 0.5) * 20,

        this.speed

    );

}

}

Prey.ts:

/**

* Prey.ts

* Класс травоядного, наследуется от Animal

*

* Особенности поведения травоядного:

* - Питается травой (Grass)

```

- * - Двигается к ближайшей доступной траве
- * - При контакте с травой съедает её и восполняет энергию
- * - При отсутствии травы движется случайным образом
- * - Имеет свои параметры: максимальный возраст 5000, энергия размножения 100
- * - Цвет отображается в синих тонах (меняется от стадии и энергии)
- */

```

import { Grass } from "./Grass";
import { Animal } from "./Animal";
import { EventLog } from "../utils/EventLog";

export class Prey extends Animal {
    // Статические параметры для всех травоядных
    static MAX_AGE = 5000;           // Максимальный возраст (тиков)
    static REPRODUCTION_ENERGY = 100; // Энергия, необходимая для размножения

    /**
     * Конструктор травоядного
     * @param x - начальная координата X
     * @param y - начальная координата Y
     * @param speed - базовая скорость (по умолчанию 1.5)
     */
    constructor(x: number, y: number, speed: number = 1.5) {
        super(x, y, speed, 50); // Начальная энергия = 50
    }

    // ===== РЕАЛИЗАЦИЯ АБСТРАКТНЫХ МЕТОДОВ ANIMAL =====

```

```
getTypeName(): string {  
    return "Prey";  
}
```

```
getEmoji(): string {  
    return "🐼";  
}
```

```
getMaxAge(): number {  
    return Prey.MAX_AGE;  
}
```

```
getReproductionEnergy(): number {  
    return Prey.REPRODUCTION_ENERGY;  
}
```

```
getInitialEnergy(): number {  
    return 50;  
}
```

```
getEnergyDecayRate(): number {  
    return 0.05;    // Потеря 0.05 энергии за тик  
}
```

```
getReproductionEnergyCost(): number {  
    return 40;    // Тратит 40 энергии на размножение  
}
```

```

getEatEnergyGain(): number {
    return 20;          // Получает 20 энергии за съеденную траву
}

/**
 * Границы стадий жизни для травоядного
 * Young: 0-1000 тиков
 * Adult: 1000-3000 тиков
 * Old: 3000+ тиков
 */
getStageBoundaries(): { young: number; adult: number } {
    return { young: 1000, adult: 3000 };
}

/**
 * Множители скорости в зависимости от стадии
 * Young: быстрее на 30%
 * Adult: нормальная скорость
 * Old: медленнее на 30%
 */
getSpeedMultipliers(): { young: number; adult: number; old: number } {
    return { young: 1.3, adult: 1.0, old: 0.7 };
}

/**
 * Множители размера в зависимости от стадии
 * Young: 80% от базового
 * Adult: 120% от базового (самые крупные)

```

* Old: 100% от базового

*/

getSizeMultipliers(): { young: number; adult: number; old: number } {

return { young: 0.8, adult: 1.2, old: 1.0 };

}

// ===== СПЕЦИФИЧЕСКИЕ МЕТОДЫ ТРАВЯДНОГО =====

/**

* Возвращает цвет травядного для отрисовки

* Цвет зависит от стадии и уровня энергии

* - Young: светлый синий (rgba(96, 165, 250))

* - Adult: насыщенный синий (rgba(59, 130, 246))

* - Old: темный синий (rgba(30, 64, 175))

* Прозрачность увеличивается при высокой энергии

*/

getColor(): string {

const stage = this.getStage();

const energyPercent = this.energy / 100; // Максимальная энергия 100

if (stage === "Young") {

return `rgba(96, 165, 250, \${0.6 + energyPercent \* 0.4})`;

}

if (stage === "Adult") {

return `rgba(59, 130, 246, \${0.7 + energyPercent \* 0.3})`;

}

return `rgba(30, 64, 175, \${0.4 + energyPercent \* 0.3})`;

}

```

/**
 * Основной метод обновления состояния травоядного
 * Вызывается каждый тик симуляции
 * @param grasses - массив всей травы на поле
 * @param worldWidth - ширина поля (для границ)
 * @param worldHeight - высота поля (для границ)
 */
update(grasses: Grass[], worldWidth: number, worldHeight: number) {
  if (!this.alive) return;

  // Базовое обновление (возраст, энергия)
  this.updateBase();

  // Поиск ближайшей доступной травы
  const target = this.findNearestGrass(grasses);

  if (target) {
    // Движение к траве
    this.moveTowards(target.x, target.y, worldWidth, worldHeight);

    // Проверка контакта с травой (дистанция < 6 пикселей)
    const distance = Math.hypot(target.x - this.x, target.y - this.y);
    if (distance < 6 && target.isAvailable) {
      // Поедание травы
      target.eat();

      this.energy += this.getEatEnergyGain();

      if (this.energy > 100) this.energy = 100; // Ограничение максимальной энергии
    }
  }
}

```



```

    }
  } else {
    // Если травы нет - случайное блуждание
    this.moveRandom(worldWidth, worldHeight);
  }

  // Проверка условий смерти (голод или старость)
  this.checkDeath();
}

/**
 * Поиск ближайшей доступной травы
 * Используется для навигации травоядного
 * @param grasses - массив всей травы
 * @returns ближайший объект Grass (доступный) или null, если травы нет
 */
private findNearestGrass(grasses: Grass[]): Grass | null {
  // Фильтруем только доступную для еды траву
  const available = grasses.filter(g => g.isAvailable);
  if (available.length === 0) return null;

  // Поиск с минимальным евклидовым расстоянием
  let nearest = available[0];
  let minDistance = Infinity;

  for (const grass of available) {
    const distance = Math.hypot(grass.x - this.x, grass.y - this.y);
    if (distance < minDistance) {

```

```

    minDistance = distance;

    nearest = grass;

}

}

return nearest;

}

/**
 * Размножение травоядного
 * Создает нового травоядного рядом с родителем
 * @returns новый объект Prey
 */
reproduce(): Prey {
    // Тратим энергию на размножение
    this.energy -= this.getReproductionEnergyCost();

    // Логируем событие
    const log = EventLog.getInstance();
    log.addEvent(`🐇 Prey #${this.id} reproduced! New Prey will be born`, "birth");

    // Создаем потомка со случайным смещением до 20 пикселей
    return new Prey(
        this.x + (Math.random() - 0.5) * 20,
        this.y + (Math.random() - 0.5) * 20,
        this.speed
    );
}

```

```
}
```

SeasonManager.ts:

```
/**
```

```
 * SeasonManager.ts
```

```
 * Класс для управления сезонами года в симуляции
```

```
 *
```

```
 * Реализует:
```

```
 * - Циклическую смену 4 сезонов: весна → лето → осень → зима → весна
```

```
 * - Влияние сезонов на скорость роста травы (Grass)
```

```
 * - Визуальное отображение текущего сезона на UI
```

```
 * - Логирование смены сезонов в EventLog
```

```
 *
```

```
 * Влияние сезонов на рост травы:
```

```
 * - Весна (множитель 1.5): трава растет быстро
```

```
 * - Лето (множитель 1.2): трава растет нормально
```

```
 * - Осень (множитель 0.8): трава растет медленно
```

```
 * - Зима (множитель 0.3): трава растет очень медленно
```

```
 */
```

```
import { EventLog } from "../utils/EventLog";
```

```
// Тип для представления сезона года
```

```
export type Season = "spring" | "summer" | "autumn" | "winter";
```

```
export class SeasonManager {
```

```
    // Текущий сезон (начинается с весны)
```

```
    private currentSeason: Season = "spring";
```

```
// Длительность сезона в тиках (600 тиков = ~18 секунд при 30мс/тик)
```

```
private ticksPerSeason: number = 600;
```

```
// Счетчик тиков текущего сезона
```

```
private tickCounter: number = 0;
```

```
/**
```

```
 * Возвращает множитель скорости роста травы для текущего сезона
```

```
 * Используется в Grass.eat() для расчета времени восстановления
```

```
 */
```

```
getGrassGrowthMultiplier(): number {
```

```
    switch (this.currentSeason) {
```

```
        case "spring": return 1.5; // Весна - бурный рост
```

```
        case "summer": return 1.2; // Лето - хороший рост
```

```
        case "autumn": return 0.8; // Осень - замедление роста
```

```
        case "winter": return 0.3; // Зима - очень медленный рост
```

```
    }
```

```
}
```

```
/**
```

```
 * Возвращает множитель энергии для животных
```

```
 * (Зарезервировано для будущего расширения функциональности)
```

```
 */
```

```
getEnergyMultiplier(): number {
```

```
    switch (this.currentSeason) {
```

```
        case "spring": return 1.1;
```

```
        case "summer": return 1.0;
```

```

    case "autumn": return 0.9;

    case "winter": return 0.7;

    }

}

/**
 * Обновление состояния сезона (вызывается каждый тик из World)
 * Увеличивает счетчик и при достижении лимита меняет сезон
 */
update() {
    this.tickCounter++;

    if (this.tickCounter >= this.ticksPerSeason) {
        this.tickCounter = 0;
        this.nextSeason();
    }
}

/**
 * Переключение на следующий сезон
 * Вызывается автоматически при достижении ticksPerSeason
 */
private nextSeason() {
    const seasons: Season[] = ["spring", "summer", "autumn", "winter"];
    const currentIndex = seasons.indexOf(this.currentSeason);
    const newSeason = seasons[(currentIndex + 1) % 4]; // Циклический переход

    // Логируем смену сезона
    const log = EventLog.getInstance();

```

```
log.addEvent('🌍 Season changed: ${this.getSeasonName()} →  
${this.getSeasonNameForLog(newSeason)}`, "season");
```

```
this.currentSeason = newSeason;
```

```
}
```

```
/**
```

```
 * Вспомогательный метод для получения названия сезона с эмодзи для лога
```

```
 * @param season - сезон
```

```
 * @returns название с эмодзи (например, "🌸 Spring")
```

```
 */
```

```
private getSeasonNameForLog(season: Season): string {
```

```
    const names = {
```

```
        spring: "🌸 Spring",
```

```
        summer: "☀ Summer",
```

```
        autumn: "🍁 Autumn",
```

```
        winter: "❄ Winter"
```

```
    };
```

```
    return names[season];
```

```
}
```

```
/**
```

```
 * Возвращает текущий сезон в виде строкового литерала
```

```
 */
```

```
getCurrentSeason(): Season {
```

```
    return this.currentSeason;
```

```
}
```

```

/**
 * Возвращает название текущего сезона с эмодзи для отображения в UI
 */
getSeasonName(): string {
    const names = {
        spring: "🌸 Spring",
        summer: "☀ Summer",
        autumn: "🍁 Autumn",
        winter: "❄ Winter"
    };
    return names[this.currentSeason];
}

/**
 * Возвращает цвет, ассоциированный с текущим сезоном
 * Используется для визуальной индикации на StatisticsPanel и SimulationCanvas
 */
getSeasonColor(): string {
    const colors = {
        spring: "#22c55e", // Зеленый (цвет новой листвы)
        summer: "#f59e0b", // Оранжевый (солнечный)
        autumn: "#f97316", // Темно-оранжевый (листва)
        winter: "#60a5fa"  // Синий (холод, снег)
    };
    return colors[this.currentSeason];
}
}

```

World.ts:

```
/**
```

```
 * World.ts
```

```
 * Главный класс, управляющий всем миром симуляции
```

```
 *
```

```
 * Отвечает за:
```

```
 * - Хранение всех объектов (трава, травоядные, хищники)
```

```
 * - Обновление состояния мира на каждом тике
```

```
 * - Управление сезонами
```

```
 * - Инициализацию и сброс симуляции
```

```
 * - Проверку условий завершения
```

```
 */
```

```
import { Grass } from "./Grass";
```

```
import { Predator } from "./Predator";
```

```
import { Prey } from "./Prey";
```

```
import { SeasonManager } from "./SeasonManager";
```

```
export class World {
```

```
    // Размеры игрового поля (в пикселях)
```

```
    width: number;
```

```
    height: number;
```

```
    // Базовые скорости животных (передаются из UI)
```

```
    preySpeed: number;
```

```
    predatorSpeed: number;
```

```
    // Время симуляции (в условных единицах)
```



```

time: number = 0;

// Флаг завершения симуляции (когда один из видов вымер)
isFinished: boolean = false;

// Массивы объектов мира
grasses: Grass[] = [];
preys: Prey[] = [];
predators: Predator[] = [];

// Менеджер сезонов (влияет на рост травы)
seasonManager: SeasonManager = new SeasonManager();

/**
 * Конструктор мира
 * @param width - ширина поля (800px)
 * @param height - высота поля (600px)
 * @param preyCount - начальное количество травоядных
 * @param predatorCount - начальное количество хищников
 * @param preySpeed - базовая скорость травоядных
 * @param predatorSpeed - базовая скорость хищников
 */
constructor(
    width: number,
    height: number,
    preyCount: number,
    predatorCount: number,
    preySpeed: number,

```

```

    predatorSpeed: number
  ) {
    this.width = width;
    this.height = height;
    this.preySpeed = preySpeed;
    this.predatorSpeed = predatorSpeed;
    this.initialize(preycount, predatorCount);
  }

  /**
   * Инициализация (или сброс) мира
   * Создает траву, травоядных и хищников со случайными координатами
   * @param preycount - количество травоядных
   * @param predatorCount - количество хищников
   */
  initialize(preycount: number, predatorCount: number) {
    // Очищаем все массивы
    this.grasses = [];
    this.preys = [];
    this.predators = [];

    // СОЗДАНИЕ ТРАВЫ
    // 150 точек травы, равномерно распределенных по полю
    for (let i = 0; i < 150; i++) {
      const grass = new Grass(
        Math.random() * this.width,
        Math.random() * this.height
      );

```

```
// Передаем менеджер сезонов для учета влияния на рост
grass.setSeasonManager(this.seasonManager);
this.grasses.push(grass);
}
```

```
// СОЗДАНИЕ ТРАВЯДНЫХ
```

```
for (let i = 0; i < preyCount; i++) {
  this.preys.push(
    new Prey(
      Math.random() * this.width,
      Math.random() * this.height,
      this.preySpeed
    )
  );
}
```

```
// СОЗДАНИЕ ХИЩНИКОВ
```

```
for (let i = 0; i < predatorCount; i++) {
  this.predators.push(
    new Predator(
      Math.random() * this.width,
      Math.random() * this.height,
      this.predatorSpeed
    )
  );
}
```

```
// Сброс времени симуляции
```

```

this.time = 0;
}

/**
 * Основной метод обновления мира (вызывается каждый тик)
 * Последовательность действий:
 * 1. Увеличение времени симуляции
 * 2. Обновление сезона
 * 3. Обновление травы (восстановление съеденной)
 * 4. Обновление всех животных (движение, еда, размножение)
 * 5. Удаление мертвых животных
 * 6. Добавление новорожденных
 * 7. Проверка условий завершения
 */
update() {
    // 1. Увеличиваем время (шаг 0.03, округление до 2 знаков)
    this.time = +(this.time + 0.03).toFixed(2);

    // 2. Обновляем сезон (смена каждые ~18 секунд)
    this.seasonManager.update();

    // 3. Обновляем траву (восстановление после поедания)
    // Передаем флаг isFinished, чтобы на паузе трава не росла
    this.grasses.forEach(g => g.update(this.isFinished));

    // 4.1. Обновление травоядных и сбор новорожденных
    const newPreys: Prey[] = [];
    for (const prey of this.preys) {

```

```

prey.update(this.grasses, this.width, this.height);

// Если травоядный жив и может размножаться - создаем потомка
if (prey.alive && prey.canReproduce()) {
    newPreys.push(prey.reproduce());
}
}

// 4.2. Обновление хищников и сбор новорожденных
const newPredators: Predator[] = [];
for (const predator of this.predators) {
    predator.update(this.preys, this.width, this.height);
    if (predator.alive && predator.canReproduce()) {
        newPredators.push(predator.reproduce());
    }
}

// 5. Удаляем мертвых животных (фильтрация массивов)
this.preys = this.preys.filter(prey => prey.alive);
this.predators = this.predators.filter(predator => predator.alive);

// 6. Добавляем новорожденных в популяции
this.preys.push(...newPreys);
this.predators.push(...newPredators);

// 7. Проверка условий завершения симуляции
if (this.isFinished) return;

// Если один из видов полностью вымер - симуляция завершается

```

```

if (this.preys.length === 0 || this.predators.length === 0) {
    this.isFinished = true;
    return;
}
}

// ===== МЕТОДЫ ДЛЯ ПОЛУЧЕНИЯ ИНФОРМАЦИИ О СЕЗОНЕ =====

getCurrentSeason() {
    return this.seasonManager.getCurrentSeason();
}

getSeasonName() {
    return this.seasonManager.getSeasonName();
}

getSeasonColor() {
    return this.seasonManager.getSeasonColor();
}

/**
 * Сброс мира с новыми параметрами
 * @param preyCount - новое количество травоядных
 * @param predatorCount - новое количество хищников
 */
reset(preyCount: number, predatorCount: number) {
    this.initialize(preyCount, predatorCount);
}

```

```
}
```

EventLog.ts:

```
/**
```

```
 * EventLog.ts
```

```
 * Класс для централизованного логирования событий в симуляции
```

```
 * Реализован с использованием паттерна Singleton
```

```
 *
```

```
 * Singleton гарантирует, что во всей программе существует только
```

```
 * один экземпляр EventLog, к которому могут обращаться все объекты
```

```
 *
```

```
 * Логируемые события:
```

```
 * - birth (рождение) - зеленый цвет в UI
```

```
 * - death (смерть) - красный цвет в UI
```

```
 * - eat (поедание) - оранжевый цвет в UI
```

```
 * - season (смена сезона) - синий цвет в UI
```

```
 * - info (информация) - серый цвет в UI
```

```
 */
```

```
// Тип события для логирования
```

```
export type LogEvent = {
```

```
  id: number;           // Уникальный номер события (для key в React)
```

```
  timestamp: number;    // Время события (Unix timestamp)
```

```
  message: string;      // Текст события
```

```
  type: "birth" | "death" | "eat" | "season" | "info"; // Тип для цветовой кодировки
```

```
};
```

```
export class EventLog {
```

```
  // Единственный экземпляр класса (Singleton)
```

```

private static instance: EventLog;

// Массив всех событий (новые добавляются в начало)
private events: LogEvent[] = [];

// Максимальное количество хранимых событий (100)
private maxSize: number = 100;

// Счетчик для генерации уникальных ID событий
private nextId: number = 1;

// Список подписчиков (компоненты React для автоматического обновления)
private listeners: (() => void)[] = [];

/**
 * Приватный конструктор (для Singleton)
 * Предотвращает создание экземпляров через new EventLog()
 */
private constructor() {}

/**
 * Получение единственного экземпляра класса
 * Если экземпляра нет - создает его
 * @returns экземпляр EventLog
 */
static getInstance(): EventLog {
  if (!EventLog.instance) {
    EventLog.instance = new EventLog();
  }
}

```



```

    }

    return EventLog.instance;
}

/**
 * Добавление нового события в лог
 * @param message - текст события
 * @param type - тип события (для цветовой кодировки)
 */
addEvent(message: string, type: LogEvent["type"] = "info") {
    // Создаем объект события
    const event: LogEvent = {
        id: this.nextId++,
        timestamp: Date.now(),
        message,
        type,
    };

    // Добавляем в начало массива (чтобы новые события были сверху)
    this.events.unshift(event);

    // Если превышен максимальный размер - удаляем самое старое событие
    if (this.events.length > this.maxSize) {
        this.events.pop();
    }

    // Уведомляем всех подписчиков об обновлении
    this.notifyListeners();
}

```

```
}
```

```
/**
```

```
 * Получение всех событий
```

```
 * @returns копия массива событий
```

```
 */
```

```
getEvents(): LogEvent[] {
```

```
    return [...this.events];
```

```
}
```

```
/**
```

```
 * Очистка лога событий
```

```
 */
```

```
clear() {
```

```
    this.events = [];
```

```
    this.notifyListeners();
```

```
}
```

```
/**
```

```
 * Подписка на обновления лога
```

```
 * @param listener - функция, которая будет вызвана при добавлении события
```

```
 * @returns функция для отписки
```

```
 */
```

```
subscribe(listener: () => void) {
```

```
    this.listeners.push(listener);
```

```
    // Возвращаем функцию отписки (для cleanup в useEffect)
```

```
    return () => {
```

```
        this.listeners = this.listeners.filter(l => l !== listener);
```

```

    };
  }

  /**
   * Уведомление всех подписчиков об изменении лога
   */
  private notifyListeners() {
    this.listeners.forEach(listener => listener());
  }
}

```

App.tsx:

```

/**
 * App.tsx
 * Корневой компонент приложения
 *
 * Отвечает за:
 * - Организацию общей структуры интерфейса (3 колонки)
 * - Управление состоянием настроек (количество и скорость животных)
 * - Подключение хука useSimulation
 * - Компоновку всех дочерних компонентов
 */

import { useState } from "react";
import SimulationCanvas from "../components/SimulationCanvas";
import ControlPanel from "../components/ControlPanel";
import SettingsPanel from "../components/SettingsPanel";
import StatisticsPanel from "../components/StatisticsPanel";

```

```

import EventLogPanel from "../components/EventLogPanel";

import { useSimulation } from "../hooks/useSimulation";

function App() {

  // ===== СОСТОЯНИЕ НАСТРОЕК =====

  // Количество животных

  const [preyCount, setPreyCount] = useState(30);    // Травоядные (по умолчанию 30)
  const [predatorCount, setPredatorCount] = useState(5); // Хищники (по умолчанию 5)


  // Скорость животных

  const [preySpeed, setPreySpeed] = useState(1.5);    // Скорость травоядных
  const [predatorSpeed, setPredatorSpeed] = useState(2); // Скорость хищников


  // ===== ПОДКЛЮЧЕНИЕ ХУКА СИМУЛЯЦИИ =====

  // Хук управляет миром, запуском/паузой/сбросом

  const { world, isRunning, start, pause, reset } = useSimulation(
    preyCount,    // Передаем текущие настройки в хук
    predatorCount,
    preySpeed,
    predatorSpeed
  );

  return (
    // Основной контейнер на весь экран (фиксированное позиционирование)
    <div
      style={{
        position: "fixed",    // Фиксация относительно окна браузера
        top: 0,

```

```

    left: 0,
    right: 0,
    bottom: 0,
    display: "flex",
    justifyContent: "center", // Горизонтальное центрирование
    alignItems: "center",    // Вертикальное центрирование
    background: "#0f172a",   // Темно-синий фон
    overflow: "hidden",      // Скрываем скроллы
  }}
>

{/* Контейнер трех колонок */}
<div
  style={{
    display: "flex",
    gap: "20px",              // Расстояние между колонками
    alignItems: "flex-start", // Выравнивание по верхнему краю
    justifyContent: "center",  // Центрирование по горизонтали
    transform: "translateY(20px)", // Небольшой сдвиг вниз
  }}
>

{/* ===== ЛЕВАЯ КОЛОНКА - СТАТИСТИКА ===== */}
<div style={{ minWidth: "200px" }}>
  <StatisticsPanel world={world} />
  {/* Отображает: сезон, время, количество Prey и Predator */}
</div>

{/* ===== ЦЕНТРАЛЬНАЯ КОЛОНКА - СИМУЛЯЦИЯ ===== */}
<div style={{ display: "flex", flexDirection: "column", alignItems: "center", gap: "16px" }}>

```

```

{ /* ЗАГОЛОВОК С ГРАДИЕНТОМ */ }

<h1
  style={{
    margin: 0,
    fontSize: "32px",
    fontWeight: "bold",
    background: "linear-gradient(135deg, #22c55e, #f59e0b, #ef4444)",
    backgroundClip: "text",
    WebkitBackgroundClip: "text",
    color: "transparent", // Текст становится прозрачным, виден градиент
    letterSpacing: "2px",
  }}
>

  Predator–Prey Simulation

</h1>

{ /* ПОЛЕ СИМУЛЯЦИИ (canvas 800x600) */ }

<SimulationCanvas world={world} />

{ /* Здесь отрисовываются все объекты и обрабатываются клики */ }

{ /* ПАНЕЛЬ УПРАВЛЕНИЯ (Start, Pause, Reset) */ }

<ControlPanel
  isRunning={isRunning}
  onStart={start}
  onPause={pause}
  onReset={reset}
/>

```

</div>

{/* ===== ПРАВАЯ КОЛОНКА - НАСТРОЙКИ + ЛОГ ===== */}

<div style={{ display: "flex", flexDirection: "column", gap: "16px", minWidth: "340px" }}>

{/* ПАНЕЛЬ НАСТРОЕК (ползунки) */}

<SettingsPanel

preyCount={preyCount}

predatorCount={predatorCount}

preySpeed={preySpeed}

predatorSpeed={predatorSpeed}

setPreyCount={setPreyCount}

setPredatorCount={setPredatorCount}

setPreySpeed={setPreySpeed}

setPredatorSpeed={setPredatorSpeed}

/>

{/* ПАНЕЛЬ ЛОГА СОБЫТИЙ */}

<EventLogPanel />

</div>

</div>

</div>

);

}

export default App;

index.css:

```
/**
```

```
* index.css
```

```
* Глобальные стили приложения
```

```
*
```

```
* Основная задача: полное отключение скроллов на всей странице
```

```
* Использует !important для гарантированного применения стилей
```

```
*/
```

```
/* Сброс стандартных отступов у всех элементов */
```

```
* {
```

```
margin: 0;
```

```
padding: 0;
```

```
box-sizing: border-box;
```

```
}
```

```
/* HTML - корневой элемент страницы */
```

```
html {
```

```
overflow: hidden !important; /* Скрываем скролл */
```

```
height: 100% !important; /* Растягиваем на всю высоту */
```

```
}
```

```
/* BODY - основной контейнер страницы */
```

```
body {
```

```
overflow: hidden !important; /* Скрываем скролл */
```

```
height: 100% !important; /* Растягиваем на всю высоту */
```

```
width: 100% !important; /* Растягиваем на всю ширину */
```

```
position: fixed !important; /* Фиксированное позиционирование */
```



```

top: 0 !important;      /* Прижимаем к верхнему краю */
left: 0 !important;     /* Прижимаем к левому краю */
right: 0 !important;    /* Прижимаем к правому краю */
bottom: 0 !important;   /* Прижимаем к нижнему краю */
background: #0f172a;    /* Темно-синий фон (как в App) */
font-family: 'Segoe UI', 'Arial', sans-serif; /* Шрифт системы */
}

```

/* ROOT - контейнер React приложения */

```

#root {
  overflow: hidden !important; /* Скрываем скролл */
  height: 100% !important;    /* Растягиваем на всю высоту */
  width: 100% !important;     /* Растягиваем на всю ширину */
  position: fixed !important;  /* Фиксированное позиционирование */
  top: 0;                     /* Прижимаем к верху */
  left: 0;                    /* Прижимаем к левому краю */
}

```

main.tsx:

/**

* main.tsx

* Точка входа в React-приложение

*

* Отвечает за:

* - Монтирование корневого компонента App в DOM-элемент #root

* - Оборачивание приложения в StrictMode для дополнительных проверок

* - Импорт глобальных стилей

*/

```

import React from "react";

import ReactDOM from "react-dom/client";

import App from "./App";

import "./index.css"; // Импорт глобальных стилей (отключает скроллы)

/**
 * Создание корневого React-элемента и рендер приложения
 *
 * document.getElementById("root") - находит элемент с id="root" в index.html
 * ReactDOM.createRoot() - создает корневой узел React 18+
 * render() - отображает компонент App внутри корневого узла
 * React.StrictMode - режим строгой проверки (помогает найти потенциальные проблемы)
 */

ReactDOM.createRoot(
  document.getElementById("root")!
).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>
);

```

